

Module Programmation I: Le Langage C

**Faculté des Sciences
Senglalia- Marrakech**

Pr. Hajar LAZAR

Hajar.lazar@uca.ac.ma

Plan

- Introduction
- Types de base, Opérateurs et Expressions
- Lecture & écriture des données
- Structures de contrôle
- Tableaux & Chaînes de caractère
- Pointeurs
- Fonctions
- Types structures, unions et synonymes



Introduction

Langages informatiques

- Un langage informatique est un outil permettant de donner des ordres (instructions) à la machine

A chaque instruction correspond une action du processeur

- Intérêt : écrire des programmes (suite consécutive d'instructions) destinés à effectuer une tâche donnée

Exemple: un programme de gestion de comptes bancaires

- Contrainte: être compréhensible par la machine

Langage machine

- Langage binaire: l'information est exprimée et manipulée sous forme d'une suite de bits
- Un bit (binary digit) = 0 ou 1 (2 états électriques)
- Une combinaison de 8 bits = 1 Octet possibilités qui permettent de coder tous les caractères alphabétiques, numériques, et symboles tels que ?, *, &, ...

Le code ASCII (American Standard Code for Information Interchange) donne les correspondances entre les caractères alphanumériques et leurs représentation binaire, Ex. A= 01000001, ?=00111111

- Les opérations logiques et arithmétiques de base (addition, multiplication, ...) sont effectuées en binaire.

Langage machine

- 🔍 **Comment communiquer avec notre processeur sans avoir à apprendre son langage binaire ?**

La solution retenue a été de créer des **langages de programmation plus évolués** que le langage machine, plus faciles à apprendre et de fournir le **traducteur** qui permettra la conversion en langage machine.

Langages de programmation évolués

Les langages **évolués** masquent la complexité de la programmation et permettent aux programmeurs d'écrire des programmes clairs et lisibles.

Les instructions écrites dans ces langages doivent être compilées pour générer des programmes exécutables et compréhensibles par le processeur.

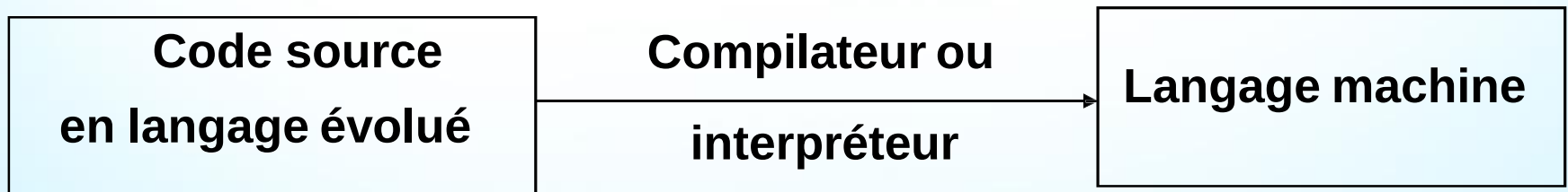
Exemples: les langages procéduraux, les langages orientés objet, etc.

Langages de programmation

- Deux types de langages:
 - Langages procéduraux: le processus de programmation est défini comme le développement d'une séquence de commandes qui manipulent des données pour produire le résultat souhaité.
 - Langages orientés objets: Le paradigme orienté objets (OOP) voit les unités de données comme objets "actifs", contrairement aux unités passives vues par le paradigme procédural.
- Exemples de langages:
 - **Fortran, Cobol, Pascal, C, ...**
 - **C++, Java, ...**

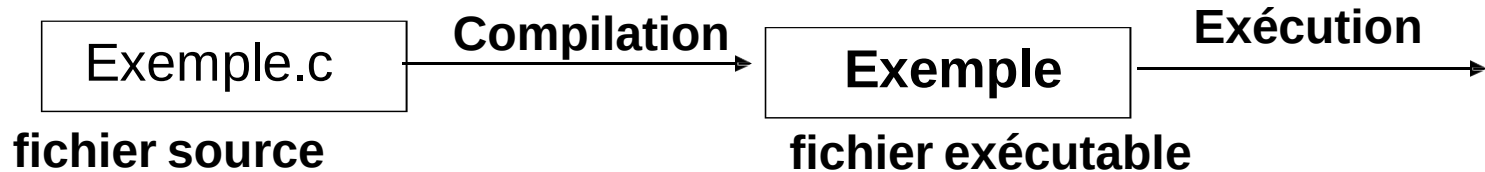
Langages haut niveau

- Intérêts multiples pour le haut niveau:
 - proche du langage humain «anglais» (compréhensible)
 - permet une plus grande portabilité (indépendant du matériel)
 - Manipulation de données et d'expressions complexes (réels, objets, $a*b/c$, ...)
- Nécessité d'un traducteur (compilateur/interpréteur),
exécution plus ou moins lente selon le traducteur



Compilateur/interpréteur

- Compilateur: traduire le programme entier une fois pour toutes



- + plus rapide à l'exécution
- + sécurité du code source
- il faut recompiler à chaque modification

- Interpréteur: traduire au fur et à mesure les instructions du programme à chaque exécution



- + exécution instantanée appréciable pour les débutants
- exécution lente par rapport à la compilation

Exemple de traduction d'un code en C en langage machine

```
int main(void)
{ return 0;
}
```

Et le même en langage machine:

```
01010101 01001000 10001001 11100101 01001000 11000111
11000000 00000000 00000000 00000000 00000000 11001001
11000011
```

Historique du C

- Le langage C a été conçu en 1972 dans «Bell Laboratories » par *Dennis Ritchie* avec l'objectif d'écrire un système d'exploitation (UNIX).
- En 1978, une première définition rigoureuse du langage C (*standard K&R-C*) a été réalisée par *Kernighan et Ritchie* en publiant le livre «The C Programming Language ».
- Le succès du C et l'apparition de compilateurs avec des extensions particulières ont conduit à sa normalisation.
- En 1983, l'organisme ANSI (American National Standards Institute) chargeait une commission de mettre au point une définition explicite et portable pour le langage C. Le résultat est le *standard ANSI-C*.

Caractéristiques du C

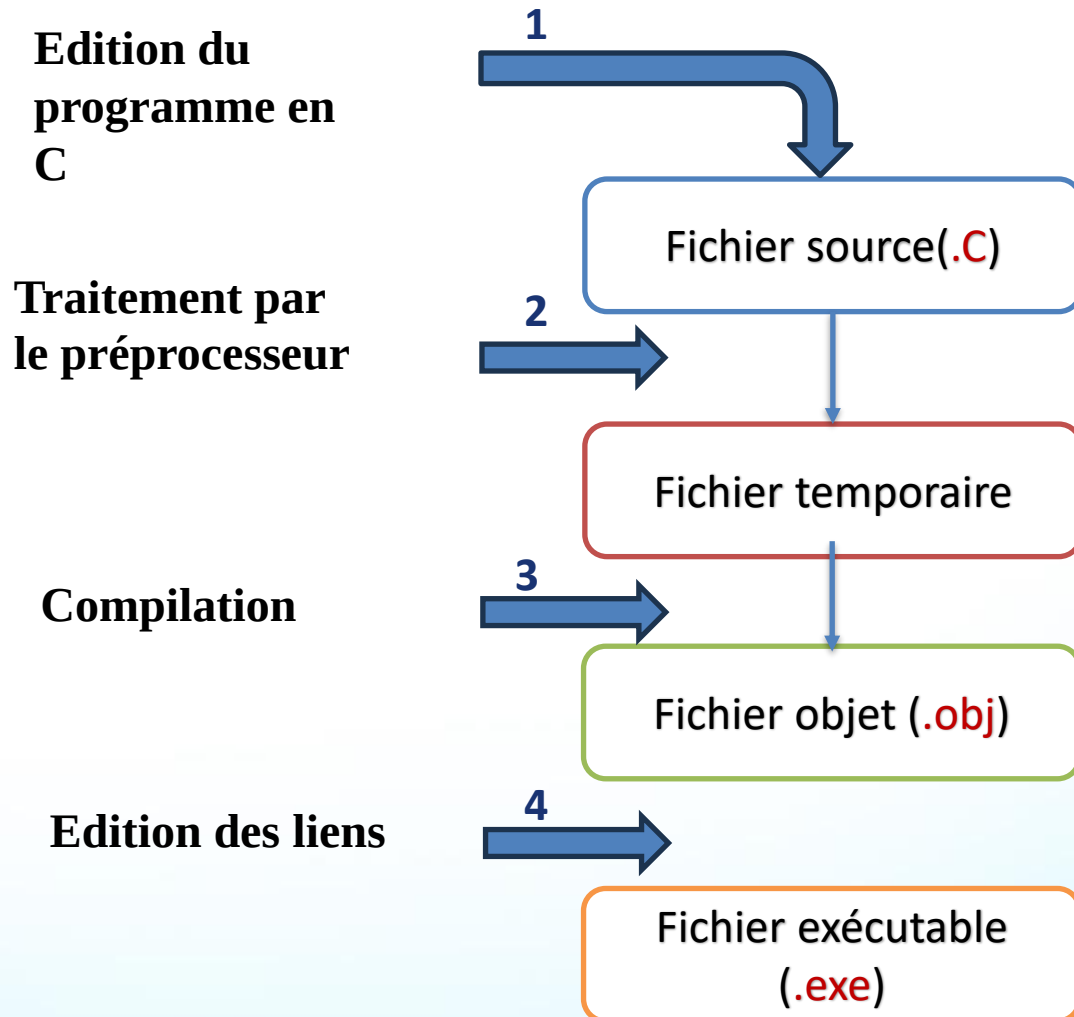
- Universel : n'est pas orienté vers un domaine d'application particulier (la robotique, finance, la gestion, ...)
- Près de la machine : offre des opérateurs qui sont proches de ceux du langage machine (manipulations de bits, d'adresses, ...) efficace
- Modulaire : peut être découpé en modules qui peuvent être compilés séparément
- Portable : en respectant le standard ANSI-C, il est possible d'utiliser le même programme sur plusieurs systèmes (hardware, système d'exploitation)

Programme source, objet et exécutable

- Un programme écrit en langage C forme un texte qu'on nomme *programme ou code source*, qui peut être formé de plusieurs fichiers sources
- Chaque fichier source est traduit par le compilateur pour obtenir un *fichier ou module objet* (formé d'instructions machine)
- Ce fichier objet n'est pas exécutable tel quel car il lui manque les instructions exécutables des fonctions standards appelées dans le fichier source (printf, scanf, ...) et éventuellement d'autres fichiers objets
- *L'éditeur de liens* réunit les différents modules objets et les fonctions de la bibliothèque standard afin de former *un programme exécutable*

Remarque : la compilation est précédée par une phase de prétraitement (inclusion de fichiers en-tête) réalisé par le *préprocesseur*

Le processus de compilation et d'exécution d'un programme en C



Composantes d'un programme C

- Directives du préprocesseur
 - inclusion des fichiers d'en-tête (fichiers avec extension .h)
 - définitions des constantes avec **#define**
- déclaration des variables globales
- définition des fonctions (En C, le programme principal et les sous-programmes sont définis comme fonctions)
- Les commentaires : texte ignoré par le compilateur, destiné à améliorer la compréhension du code

exemple : *#include<stdio.h>*

main(){

printf("notre premier programme C \n");

*/*ceci est un commentaire*/*

}

Composantes d'un programme C

- `#include<stdio.h>` fichier d'entête contenant la déclaration des fonctions d'entrées-sorties dont la fonction `printf`
- La fonction `main` est la fonction principale des programmes en C: Elle se trouve obligatoirement dans tous les programmes. L'exécution d'un programme entraîne automatiquement l'appel de la fonction `main`.
- L'appel de `printf` avec l'argument "notre premier programme C\n" permet d'afficher : notre premier programme C et \n ordonne le passage à la ligne suivante
- En C, toute instruction simple est terminée par un point-virgule ;
- Un commentaire en C est compris entre `//` et la fin de la ligne ou bien entre `/*` et `*/`

Configuration de l'environnement de développement



Code::Blocks



NetBeans



CodeLite

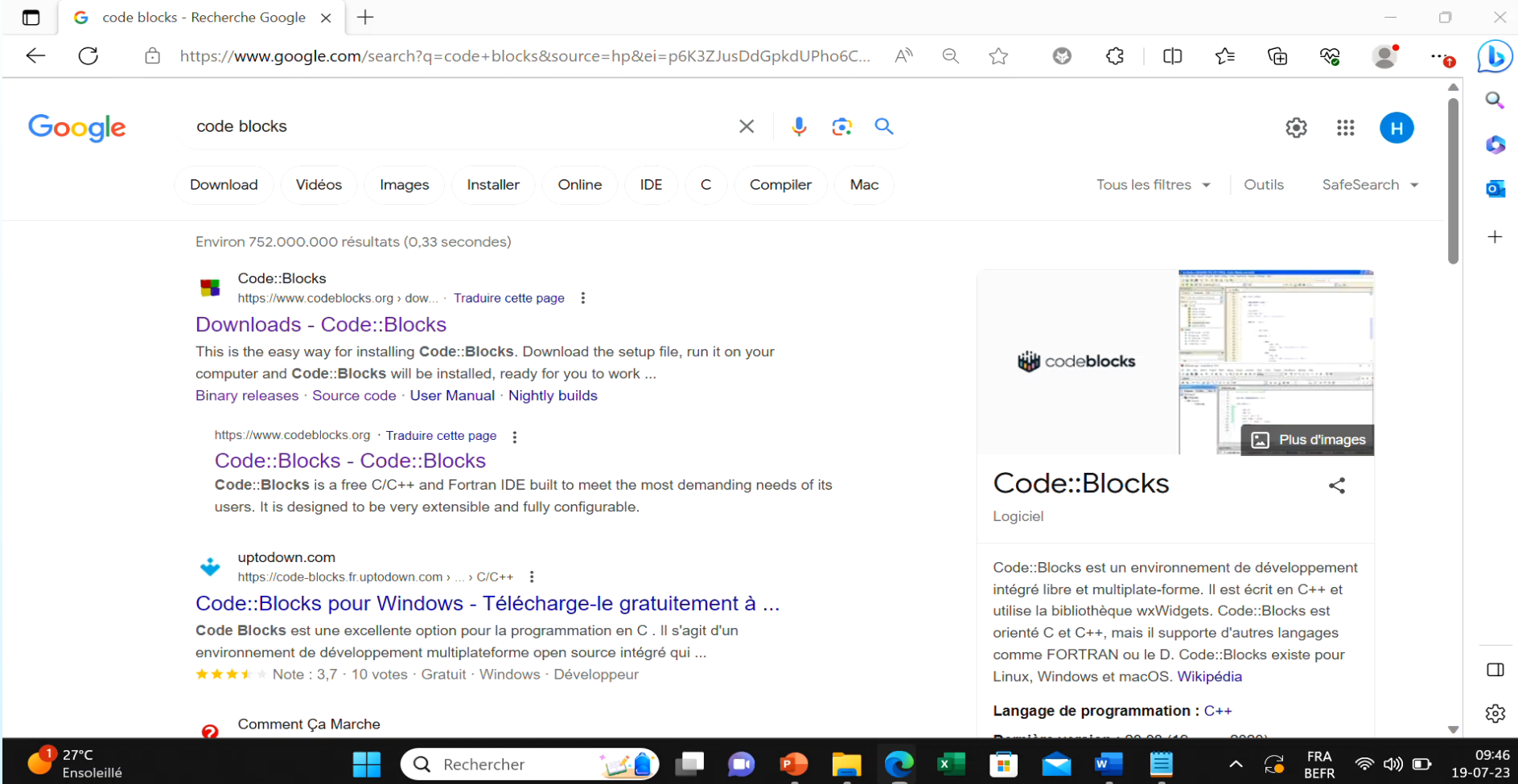


Visual Studio



Eclipse

Configuration de l'environnement de développement





Variables, types, opérateurs et expressions

Les variables

- Les variables servent à stocker les valeurs des données utilisées pendant l'exécution d'un programme
- Les variables doivent être **déclarées** avant d'être utilisées, elles doivent être caractérisées par :
 - un nom (**Identificateur**)
 - un **type** (entier, réel, ...)

(Les types de variables en C seront discutés par la suite)

Les identificateurs

Le choix d'un identificateur (nom d'une variable ou d'une fonction) est soumis à quelques règles :

- doit être constitué uniquement de lettres, de chiffres et du caractère souligné _ (Eviter les caractères de ponctuation et les espaces)
correct: `PRIX_HT, prixHT` **incorrect:** `PRIX-HT, prix HT, prix.HT`
- doit commencer par une lettre (y compris le caractère souligné)
correct : `A1, _A1` **incorrect:** `1A`
- doit être différent des mots réservés du langage :
`auto break case char const continue default do double else enum
extern float for goto if int long register return short signed
sizeof static struct switch typedef union unsigned void
volatile while`

Remarque : C distingue les majuscules et les minuscules. `NOMBRE` et `nombre` sont des identificateurs différents

Les types de base

- Le type d'une variable détermine l'ensemble des valeurs qu'elle peut prendre et le nombre d'octets à lui réserver en mémoire
- En langage C, il n'y a que deux types de base *les entiers* et *les réels* avec différentes variantes pour chaque type

Remarques:

- Un type de base est un type pour lequel une variable peut prendre une seule valeur à un instant donné contrairement aux types agrégés
- Le type caractère apparaît en C comme cas particulier du type entier (un caractère est un nombre entier, il s'identifie à son codeASCII)
- En C il n'existe pas de type spécial pour chaînes de caractères. Les moyens de traiter les chaînes de caractères seront présentés aux chapitressuivants
- Le type booléen n'existe pas. Un booléen est représenté par un entier (un entier non nul équivaut à vrai et la valeur zero équivaut à faux)

Types Entier

4 variantes d'entiers :

- **char** : caractères (entier sur 1 octet : - 128 à 127)
- **short ou short int** : entier court (entier sur 2 octets : - 32768 à 32767)
- **int** : entier standard (entier sur 2 ou 4 octets)
- **long ou long int** : entier long (4 octets : - 2147483648 à 2147483648)

Si on ajoute le préfixe **unsigned** à la définition d'un type de variables entières, alors la plage des valeurs change:

- **unsigned char** : 0 à 255
- **unsigned short** : 0 à 65535
- **unsigned int** : dépend du codage (sur 2 ou 4 octets)
- **unsigned long** : 0 à 4294967295

Types Réel

3 variantes de réels :

- **float** : réel simple précision codé sur 4 octets de
 -3.4×10^{38} à 3.4×10^{38}
- **double** : réel double précision codé sur 8 octets de
 -1.7×10^{308} à 1.7×10^{308}
- **long double** : réel très grande précision codé sur 10 octets de
 -3.4×10^{4932} à 3.4×10^{4932}

Déclaration des variables

- Les *déclarations* introduisent les variables qui seront utilisées, fixent leur type et parfois aussi leur valeur de départ (initialisation)

- Syntaxe de déclaration en C

<Type> <NomVar1>,<NomVar2>,....,<NomVarN>;

- Exemple:

int i, j,k;

float x, y ;

double z=1.5; // déclaration et initialisation

short compteur;

char c=`A`;

Déclaration des constantes

- Une constante conserve sa valeur pendant toute l'exécution d'un programme
- En C, on associe une valeur à une constante en utilisant :
 - la directive *#define* :
#define nom_constant valeur
Ici la constante ne possède pas de type.
exemple: *#define Pi 3.141592*
 - le mot clé *const* :
const type nom = expression ;
Dans cette instruction la constante est typée
exemple : *const float Pi = 3.141592*

(Rq: L'intérêt des constantes est de donner un nom parlant à une valeur, par exemple NB_LIGNES, aussi ça facilite la modification du code)

Constantes entières

On distingue 3 formes de constantes entières :

- **forme décimale** : c'est l'écriture usuelle. Ex : 372, 200
- **forme octale** (base 8) : on commence par un 0 suivi de chiffres octaux. Ex : 0477
- **forme hexadécimale** (base 16) : on commence par 0x (ou 0X) suivis de chiffres hexadécimaux (0-9 a-f). Ex : 0x5a2b, 0Xa9f

Remarques sur les constantes entières

- Le compilateur attribue automatiquement un type aux constantes entières. Il attribue en général le type le plus économique parmi (int, unsigned int, long int, unsigned long int)
- On peut forcer la machine à utiliser un type de notre choix en ajoutant les suffixes suivants:
 - u ou U pour unsigned int, Ex : 100U, 0xAu
 - l ou L pour long, Ex : 15l, 0127L
 - ul ou UL pour unsigned long, Ex : 1236UL, 035ul

Constantes réelles

On distingue 2 notations :

- notation décimale Ex : 123.4, .27, 5.
- notation exponentielle Ex : 1234e-1 ou 1234E-1

Remarques :

- Les constantes réelles sont par défaut de type double
- On peut forcer la machine à utiliser un type de notre choix en ajoutant les suffixes suivants:
 - f ou F pour le type float, Ex: 1.25f
 - l ou L pour le type long double, EX: 1.0L

Les constantes caractères

- Se sont des constantes qui désignent un seul caractère, elles sont toujours indiquées entre des apostrophes, Ex : 'b', 'A', '?'
- La valeur d'une constante caractère est le code ASCII du caractère
- Les caractères constants peuvent apparaître dans des opérations arithmétiques ou logiques
- Les constantes caractères sont de type int

Expressions et opérateurs

- Une *expression* peut être une valeur, une variable ou une opération constituée par des valeurs, des constantes et des variables reliées entre eux par des *opérateurs*

exemples: 1, b, a*2, a+ 3*b-c, ...

- Un *opérateur* est un symbole qui permet de manipuler une ou plusieurs variables pour produire un résultat. On distingue :
 - les *opérateurs binaires* qui nécessitent deux opérandes (ex : $a + b$)
 - les *opérateurs unaires* qui nécessitent un seul opérande (ex: $a++$)
 - l'*opérateur conditionnel ?:* , le seul qui nécessite trois opérandes
- Une expression fournit une seule valeur, elle est évaluée en respectant des règles de priorité et d'associativité

Opérateurs en C

- Le langage C est riche en opérateurs. Outre les opérateurs standards, il comporte des opérateurs originaux d'affectation, d'incrémentation et de manipulation de bits
- On distingue les opérateurs suivants en C :
 - **les opérateurs arithmétiques** : +, -, *, /, %
 - **les opérateurs d'affectation** : =, +=, -=, *=, /=, ...
 - **les opérateurs logiques** : &&, ||, !
 - **les opérateurs de comparaison** : ==, !=, <, >, <=, >=
 - **les opérateurs d'incrémentation et de décrémentation** : ++, --
 - **les opérateurs sur les bits** : <<, >>, &, |, ~, ^
 - **d'autres opérateurs particuliers** : ?:, sizeof, cast

Opérateurs arithmétiques

- binaires : $+$ $-$ $*$ $/$ et $\%$ (modulo) et unaire : $-$
- Les opérandes peuvent être des entiers ou des réels sauf pour $\%$ qui agit uniquement sur des entiers
- Lorsque les types des deux opérandes sont différents il y'a conversion implicite dans le type le plus fort
- L'opérateur $/$ retourne un quotient entier si les deux opérandes sont des entiers ($5 / 2 \rightarrow 2$). Il retourne un quotient réel si l'un au moins des opérandes est un réel ($5.0 / 2 \rightarrow 2.5$)

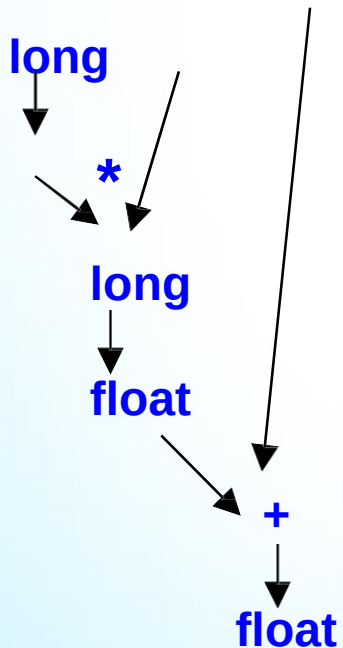
Conversions implicites

- Les types `short` et `char` sont systématiquement convertis en `int` indépendamment des autres opérandes
- La conversion se fait en général selon une hiérarchie qui n'altère pas les valeurs `int` → `long` → `float` → `double` → `long double`
- **Exemple1** : `n * x + p` (`int n,p; float x`)
 - exécution prioritaire de `n * x` : conversion de `n` en `float`
 - exécution de l'addition : conversion de `p` en `float`
- **Exemple2** : `p1 * p2 + p3 * x` (`char p1, short p2, p3 ; float x`)
 - `p1`, `p2` et `p3` d'abord convertis en `int`
 - `p3` converti en `float` avant multiplication

Exemple de conversion

Exemple : $n * p + x$ (int n ; long p ; float x)

$n * p + x$



conversion de n en long

multiplication par p

$n * p$ de type long

conversion de $n * p$ en float

addition

résultat de type float

Opérateur d'affectation simple =

- L'opérateur = affecte une valeur ou une expression à une variable
 - Exemple: double x,y,z; x=2.5; y=0.7; z=x*y-3;
- Le terme à gauche de l'affectation est appelé *lvalue* (leftvalue)
- L'affectation est interprétée comme une expression. La valeur de l'expression est la valeur affectée
- On peut enchaîner des affectations, l'évaluation se fait de droite à gauche
 - exemple : i = j = k = 5 (est équivalente à k = 5, j=k et ensuite i=j)
- La valeur affectée est toujours convertie dans le type de la lvalue, même si ce type est plus faible (ex : conversion de **float** en **int**, avec perte d'information)

Opérateurs relationnels

- **Opérateurs**

- $<$: inférieur à
- $>$: supérieur à
- $<=$: inférieur ou égal à
- $>=$: supérieur ou égal à
- $!=$: différent de
- $==$: égal à

- Le résultat de la comparaison n'est pas une valeur booléenne, mais 0 si le résultat est faux et 1 si le résultat est vrai
- Les expressions relationnelles peuvent donc intervenir dans des expressions arithmétiques
- Exemple: $a=2, b=7, c=4$
 - $b==3 \rightarrow 0$ (faux)
 - $a!=b \rightarrow 1$ (vrai)
 - $4*(a<b) + 2*(c>=b) \rightarrow 4$

Opérateurs logiques

- **&&** : ET logique **||** : OU logique **!** : négation logique
- **&&** retourne vrai si les deux opérandes sont vrais (valent 1) et 0 sinon
- **||** retourne vrai si l'une des opérandes est vrai (vaut 1) et 0 sinon
- Les valeurs numériques sont acceptées : toute valeur non nulle correspond à vraie et 0 correspond à faux
 - Exemple : $5 \ \&\& \ 11 \rightarrow 1$
 $!13.7 \rightarrow 0$

Évaluation de && et ||

- Le 2^{ème} opérande est évalué uniquement en cas de nécessité
 - `a && b` : b évalué uniquement si a vaut vrai (si a vaut faux, évaluation de b inutile car `a && b` vaut faux)
 - `a || b` : b évalué uniquement si a vaut faux (si a vaut vrai, évaluation de b inutile car `a || b` vaut vrai)
- **Exemples**
 - `if ((d != 0) && (n / d == 2))` : pas de division si d vaut 0
 - `if ((n >= 0) && (sqrt(n) < p))` : racine non calculée si n < 0
- L'intérêt est d'accélérer l'évaluation et d'éviter les traitements inappropriés

Incrémentation et décrémentation

- Les opérateurs ++ et -- sont des opérateurs unaires permettant respectivement d'ajouter et de retrancher 1 au contenu de leur opérande
- Cette opération est effectuée après ou avant l'évaluation de l'expression suivant que l'opérateur suit ou précède son opérande
 - $k = i++$ (*post-incrémentation*) affecte d'abord la valeur de i à k et incrémente après ($k = i++ ; \Leftrightarrow k = i ; i = i+1 ;$)
 - $k = ++i$ (*pré-incrémentation*) incrémente d'abord et après affecte la valeur incrémentée à k ($k = ++i ; \Leftrightarrow i = i+1 ; k = i ;$)
- Exemple

$i = 5 ; n = ++i - 5 ;$	
:	i vaut 6 et n vaut 1
$i = 5 ; n = i++ - 5 ;$	
	i vaut 6 et n vaut 0
- Remarque : idem pour l'opérateur de décrémentation --

Opérateurs de manipulations de bits

- **opérateurs arithmétiques bit à bit :**

& : ET logique **|** : OU logique **^** : OU exclusif **~** : négation

- Les opérandes sont de type entier. Les opérations s'effectuent bit à bit suivant la logique binaire

b1	b2	~b1	b1&b2	b1 b2	b1^b2
1	1	0	1	1	0
1	0	0	0	1	1
0	1	1	0	1	1
0	0	1	0	0	0

- Ex : 14= 1110 , 9=1001 → 14 & 9= 1000=8, 14 | 9 =1111=15

Opérateurs de décalage de bits

- Il existe deux opérateurs de décalage :
 >> : décalage à droite << : décalage à gauche
- L'opérande gauche constitue l'objet à décaler et l'opérande droit le nombre de bits de décalage
- Dans le cas d'un décalage à gauche les bits les plus à gauche sont perdus. Les positions binaires rendues vacantes sont remplies par des 0
- Ex : char x=14; (14=00001110) → $14 \ll 2 = 001110\mathbf{00} = 56$

Rq : un décalage à gauche de k bits correspond (sauf débordement) à la multiplication par 2^k

Opérateurs de décalage de bits

- Lors d'un décalage à droite les bits les plus à droite sont perdus.
 - si l'entier à décaler est non signé, les positions binaires rendues vacantes sont remplies par des 0
 - s'il est signé le remplissage dépend de l'implémentation (en général le remplissage se fait par le bit du signe)
- Ex : char x=14; (14=00001110) $\rightarrow 14 >> 2 =$
00000011 = 3

Opérateurs d'affectation combinés

- Soit un opérateur de calcul **op** et deux expressions *exp1* et *exp2*. L'expression *exp1 = exp1 op exp2* peut s'écrire en général de façon équivalente sous la forme *exp1 op= exp2*

- Opérateurs utilisables :

	-=	*=	/=	%=
• +=	>>=	&=	^=	=
• <<=				

- Exemples :

- $a = a + b$ s'écrit : $a += b$
- $n = n \% 2$ s'écrit : $n \% = 2$
- $x = x * i$ s'écrit : $x * = i$
- $p = p >> 3$ s'écrit : $p >> = 3$

Opérateur de forçage de type (cast)

- Il est possible d'effectuer des conversions explicites ou de forcer le type d'une expression
 - Syntaxe : `(<type>) <expression>`
 - Exemple : `int n, p ;`
`(double) (n / p);` convertit l'entier `n / p` en double
- Remarque : la conversion (ou casting) se fait après calcul
$$(\text{double}) (n/p) \neq (\text{double}) n / p \neq (\text{double}) (n) / (\text{double}) (p)$$
 - `float n = 4.6, p = 1.5 ;`
 - `(int) n / (int) p = 4 / 1 = 4`
 - `(int) n / p = 4 / 1.5 = 2.66`
 - `n / (int) p = 4.6 / 1 = 4.6`
 - `n / p = 4.6 / 1.5 = 3.06`

Opérateur conditionnel ?

- **Syntaxe:** `exp1 ? exp2 : exp3`

exp1 est évaluée, si sa valeur est non nulle c'est exp2 qui est exécutée, sinon exp3

- **Exemple1 :** `max = a > b ? a : b`

Si $a > b$ alors on affecte à max le contenu de exp2 càd a sinon on lui affecte b

- **Exemple2 :** `a > b ? i++ : i--;`

Si $a > b$ on incrémente i sinon on décrémente i

Opérateur séquentiel,

- Utilité : regrouper plusieurs sous-expressions ou calculs en une seule expression
- Les calculs sont évalués en séquence de gauche à droite
- La valeur de l'expression est celle de la dernière sous-expression
- Exemples
 - `i++ , i + j;` // on évalue `i++` ensuite `i+j` (on utilise la valeur de `i` incrémentée)
 - `i++ , j = i + k , a + b;` // la valeur de l'expression est celle de `a+b`

Opérateur SIZEOF

- **Syntaxe : `sizeof (<type>)` ou `sizeof (<variable>)`**
fournit la taille en octets d'un type ou d'une variable
- **Exemples**
 - `int n;`
 - `printf ("%d \n",sizeof(int)); // affiche 4`
 - `printf ("%d \n",sizeof(n)); // affiche 4`

Priorité et associativité des opérateurs

- Une expression est évaluée en respectant des règles de priorité et d'associativité des opérateurs
 - Ex: * est plus prioritaire que +, ainsi $2 + 3 * 7$ vaut 23 et non 35
- Le tableau de la page suivante donne la priorité de tous les opérateurs. La priorité est décroissante de haut en bas dans le tableau.
- Les opérateurs dans une même ligne ont le même niveau de priorité. Dans ce cas on applique les règles d'associativité selon le sens de la flèche. Par exemple: $13 \% 3 * 4$ vaut 4 et non 1
- Remarque: en cas de doute il vaut mieux utiliser les parenthèses pour indiquer les opérations à effectuer en priorité. Ex: $(2 + 3) * 7$ vaut 35

Priorités de tous les opérateurs

- La priorité est décroissante de haut en bas dans le tableau.
- La règle d'associativité s'applique pour tous les opérateurs d'un même niveau de priorité. (-> pour une associativité de gauche à droite et <- pour une associativité de droite à gauche). Les parenthèses forcent la priorité

Priorité 1 (la plus forte):	()	→
Priorité 2:	! ++ --	←
Priorité 3:	* / %	→
Priorité 4:	+ -	→
Priorité 5:	< <= > >=	→
Priorité 6:	== !=	→
Priorité 7:	&&	→
Priorité 9 (la plus faible):	= += -= *= /= %=	←

Exercices

- Quelle est la valeur de i après la suite d'instructions :

```
int i=10;  
i = i- (i--);
```

- Quelle est la valeur de i après la suite d'instructions :

```
int i=10;  
i = i- (--i);
```



Lecture & écriture des données

Les instructions de lecture et d'écriture

- Il s'agit des instructions permettant à la machine de dialoguer avec l'utilisateur
- Dans un sens la lecture permet à l'utilisateur d'entrer des valeurs au clavier pour qu'elles soient utilisées par le programme
- Dans l'autre sens, l'écriture permet au programme de communiquer des valeurs à l'utilisateur en les affichant à l'écran (ou en les écrivant dans un fichier)

Les instructions de lecture et d'écriture

- La bibliothèque standard <stdio> contient un ensemble de fonctions qui assurent la lecture et l'écriture des données.
- Dans ce chapitre, nous allons en discuter les plus importantes:
 - `printf()` écriture formatée de données
 - `scanf()` lecture formatée de données

Ecriture formatée de données: printf ()

- la fonction **printf** est utilisée pour afficher à l'écran du texte, des valeurs de variables ou des résultats d'expressions.
- **Syntaxe :** **printf("format", expr1, expr2, ...);**
 - **expr1,...** : sont les variables et les expressions dont les valeurs sont à représenter
 - **Format** : est une chaîne de caractères qui peut contenir
 - du texte
 - des séquences d'échappement ('\\n', '\\t',...)
 - des spécificateurs de format : un ou deux caractères précédés du symbole %, indiquant le format d'affichage

Rq : Le nombre de spécificateurs de format doit être égale au nombre d'expressions!

Spécificateurs de format

SYMBOLE	TYPE	AFFICHAGE COMME
<i>%d ou %i</i>	<i>int</i>	entier relatif
<i>%u</i>	<i>unsinged int</i>	entier naturel non signé
<i>%c</i>	<i>char</i>	caractère
<i>%o</i>	<i>int</i>	entier sous forme octale
<i>%x ou %X</i>	<i>int</i>	entier sous forme hexadécimale
<i>%f</i>	<i>float, double</i>	réel en notation décimale
<i>%e ou %E</i>	<i>float, double</i>	réel en notation exponentielle
<i>%s</i>	<i>char*</i>	chaîne de caractères

Séquences d'échappement

- l'affichage du texte peut être contrôlé à l'aide des *séquences d'échappement* :
 - **\n** : nouvelle ligne
 - **\t** : tabulation horizontale
 - **\a** : signal sonore
 - **\b** : retour arrière
 - **\r** : retour chariot
 - **\v** : tabulation verticale
 - **\f** : saut de page
 - **** : back slash (\)
 - **\'** : apostrophe
 - **\"** : guillemet

Exemples de printf()

```
#include<stdio.h>
main()
{ int i=1 , j=2, N=15;
  printf("la somme de %d et %d est %d \n", i, j, i+j);
  printf(" N= %x \n" , N);
  char c='A';
  printf(" le code Ascii de %c est %d \n", c, c);
}
```

Ce programme va afficher :

la somme de 1 et 2 est 3

N=f

le code Ascii de A est 65

Remarque : Pour pouvoir traiter correctement les arguments du type long, il faut utiliser les spécificateurs %ld, %li, %lu, %lo, %lx

Exemples de printf()

```
#include<stdio.h>
```

```
main()
```

```
{ double x=10.5, y=2.5;  
  printf("%f divisé par %f égal à %f \n", x, y, x/y);  
  printf("%e divisé par %e égal à %e\n", x, y, x/y);  
}
```

Ce programme va afficher :

10.500000 divisé par 2.500000 égal à 4.200000

1.050000e+001 divisé par 2.500000e+000 égal à 4.200000e+000

Remarque : Pour pouvoir traiter correctement les arguments du type long double, il faut utiliser les spécificateurs %Lf et %le

Remarques sur l'affichage

- Par défaut, les entiers sont affichés sans espaces avant ou après
- Pour agir sur l'affichage → un nombre est placé après % et précise le nombre de caractères **minimum à utiliser**

- Exemples : `printf("%4d", n);`

`n = 20` → `~~20` (~ : espace)

`n=56123` → `56123`

`printf("%4X", 123);` → `~~7B`

`printf("%4x", 123);` → `~~7b`

Remarques sur l'affichage

- Pour les réels, on peut préciser la *largeur minimale* de la valeur à afficher et le nombre de chiffres après le point décimal.
- La précision par défaut est fixée à six décimales. Les positions décimales sont arrondies à la valeur la plus proche.

- Exemples :

<code>printf("%f", 100.123);</code>	➔ 100.123000
<code>printf("%12f", 100.123);</code>	➔ ~~100.123000
<code>printf("%.2f", 100.123);</code>	➔ 100.12
<code>printf("%5.0f", 100.123);</code>	➔ ~~100
<code>printf("%.4f", 1.23456);</code>	➔ 1.2346

Lecture formatée de données: scanf ()

- la fonction **scanf** permet de lire des données à partir du clavier
- **Syntaxe** : **scanf**("format", AdrVar1, AdrVar2, ...);
 - **Format** : le format de lecture de données, est le même que pour *printf*
 - **adrVar1, adrVar2, ...** : adresses des variables auxquelles les données seront attribuées. L'adresse d'une variable est indiquée par le **nom** de la variable **précédé** du signe **&**

Exemples de scanf()

```
#include<stdio.h>
main()
{ int i , j;
    scanf("%d%d", &i, &j);
    printf("i=%d    et    j=%d", i, j);
}
```

ce programme permet de lire deux entiers entrés au clavier et les afficher à l'écran.

Remarque : pour lire une donnée du type **long**, il faut utiliser les spécificateurs
%ld, %li, %lu, %lo, %lx.

Exemples de scanf()

```
#include<stdio.h>
main()
{ float   x;
  double y;
  scanf("%f %lf", &x, &y);
  printf("x=%f et y=%f", x,y);
}
```

Ce programme permet de lire un réel simple et un autre double du clavier et les afficher à l'écran

Remarque : pour lire une donnée du type **double**, il faut utiliser **%le** ou **%lf** et pour lire une donnée du type **long double**, il faut utiliser **%Le** ou **%Lf**

Ecriture d'un caractère: putchar()

- `Putchar` permet d'afficher un caractère sur l'écran.
- `Putchar(c);` est équivalente à `printf("%c", c);`
- **Forme générale:** `putchar(<caractere>);`
- Elle reçoit comme argument la valeur d'un caractère convertie en entier.
- **Exemples**
`char a = 63 ;`
`char b = '\n' ;`
`putchar('x') ; /* affiche la lettre x */`
`putchar(b) ; /* retour à la ligne */`
`putchar(65) ; /* affiche le caractère de code ASCII = 65: A */`

Remarque: `putchar` retourne la valeur du caractère écrit toujours considéré comme un entier, ou bien la valeur -1 (EOF) en cas d'erreur.

Lecture d'un caractère: getchar()

- Permet de lire un caractère depuis le clavier.
- `c=getchar(c);` est équivalente à `scanf("%c",&c);`
- **Forme générale:** `<Caractere> = getchar() ;`
- Remarques:

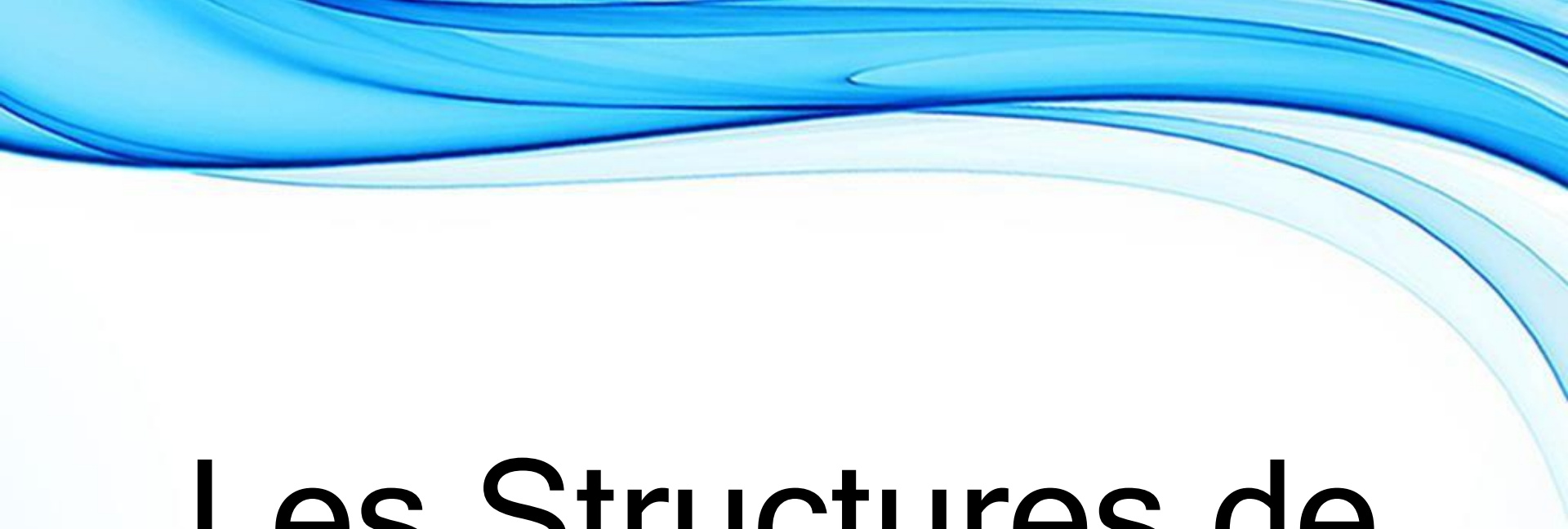
getchar retourne le caractère lu(un entier entre 0 et 255), ou bien la valeur -1 (EOF).

Getchar lit les données depuis le clavier et fournit les données après confirmation par la touche "entrée"

- Exemple :

```
int c ;
```

```
c = getchar() ; /* attend la saisie d'un caractère au clavier */
```



Les Structures de Contrôle

Structures de contrôle

- Les structures de contrôle définissent la façon avec laquelle les instructions sont effectuées. Elles conditionnent l'exécution d'instructions à la valeur d'une expression
- On distingue :
 - **Les structures alternatives (tests)** : permettent d'effectuer des choix càd de se comporter différemment suivant les circonstances (valeur d'une expression). En C, on dispose des instructions : ***if...else*** et ***switch***.
 - **Les structures répétitives (boucles)** : permettent de répéter plusieurs fois un ensemble donné d'instructions. Cette famille dispose des instructions : ***while***, ***do...while*** et ***for***.

L'instruction if...else

- **Syntaxe :** *If (expression)*
 bloc-instruction1
 else
 bloc-instruction2
- *bloc-instruction* peut être une seule instruction terminée par un point-virgule ou une suite d'instructions délimitées par des accolades{ }
- *expression* est évaluée, si elle est vraie (valeur différente de 0), alors *bloc-instruction1* est exécuté. Si elle est fausse (valeur 0) alors *bloc-instruction2* est exécuté
- La partie *else* est facultative. S'il n'y a pas de traitement à réaliser quand la condition est fausse, on utilisera simplement la forme :

If (expression)
bloc-instruction1

if...else : exemple

- *float a, b, max;*
 if (a > b)
 max = a;
 else
 max = b;

Imbrication des instructions if

- On peut imbriquer plusieurs instructions if...else
- Ceci peut conduire à des confusions, par exemple :
 - if (N>0)
 if (A>B)
 MAX=A;
 else MAX=B; (interprétation 1 : si N=0 alors MAX prend la valeur B)
 - if (N>0)
 if (A>B)
 MAX=A;
 else MAX=B; (interprétation 2 : si N=0 MAX ne change pas)
- En C un *else* est toujours associé au dernier *if* qui ne possède pas une partie *else* (c'est l'interprétation 2 qui est juste)

Imbrication des instructions if

- Conseil : pour éviter toute ambiguïté ou pour forcer une certaine interprétation dans l'imbrication des *if*, il vaut mieux utiliser les accolades

- ```
if(a<=0)
 {if(a==0)
 printf("a est nul ");
 else
 printf(" a est strictement négatif ");
 }
else
 printf(" a est strictement positif ");
```

- Pour forcer l'interprétation 1: *if (N>0)*

```
 { if (A>B)
 MAX=A;
 }
else MAX=B;
```

# L'instruction d'aiguillage switch :

- Permet de choisir des instructions à exécuter selon la valeur d'une expression qui doit être de type entier

- la syntaxe est :

```
switch (expression) {
 case expression_constante1 : instructions_1; break;
 case expression_constante2 : instructions_2; break;
 ...
 case expression_constante n : instructions_n; break;
 default : instructions;
}
```

- *expression\_constantei* doit être une expression constante **entière**
- Instructions *i* peut être une instruction simple ou composée
- *break* et *default* sont optionnels et peuvent ne pas figurer

# Fonctionnement de switch

- *expression est évaluée*
- *si sa valeur est égale à une expression\_constante i, on se branche à ce cas et on exécute les instructions\_i qui lui correspondent*
  - On exécute aussi les instructions des cas suivants jusqu'à la fin du bloc ou jusqu'à une instruction break (qui fait sortir de la structure switch)
- si la valeur de l'expression n'est égale à aucune des expressions constantes
  - Si **default** existe, alors on exécute les instructions qui le suivent
  - Sinon aucune instruction n'est exécutée



## Switch : exemple

```
main()
{ char c;
 switch (c) {
 case 'a':
 case 'e':
 case 'i':
 case 'o':
 case 'u':
 case 'y': printf("voyelle\n");break ;
 default : printf("consonne\n");
 }
}
```

# Les boucles while et do .. while

```
while (condition)
{
 instructions
}
```

```
do
{
 instructions
} while (condition);
```

- la condition (dite condition de contrôle de la boucle) est évaluée à chaque itération. Les instructions (corps de la boucle) sont exécutés tant que la condition est vraie, on sort de la boucle dès que la condition devient fausse
- dans la boucle while le test de continuation s'effectue avant d'entamer le corps de boucle qui, de ce fait, peut ne jamais s'exécuter
- par contre, dans la boucle do-while ce test est effectué après le corps de boucle, lequel sera alors exécuté au moins une fois

## Boucle while : exemple

Un programme qui détermine le premier nombre entier N tel que la somme de 1 à N dépasse strictement 100

```
main()
{ int i, som;
 i =0; som= 0;
 while (som <=100)
 {
 som+=i;
 i++;
 }
 printf (" La valeur cherchée est N= %d\n ", i);
}
```

## Boucle do .. while : exemple

Contrôle de saisie d'une note saisie au clavier jusqu'à ce que la valeur entrée soit valable

```
main()
{ int N;
 do {
 printf (" Entrez une note comprise entre 0 et 20 \n");
 scanf("%d",&N);
 } while (N < 0 || N > 20);
}
```

# La boucle for

```
for (expr1 ; expr2 ; expr3)
{
 instructions
}
```

- L'expression expr1 est évaluée une seule fois au début de l'exécution de la boucle. Elle effectue l'initialisation des données de la boucle
- L'expression expr2 est évaluée et testée avant chaque passage dans la boucle. Elle constitue le test de continuation de la boucle.
- L'expression expr3 est évaluée après chaque passage. Elle est utilisée pour réinitialiser les données de la boucle

## Boucle for : remarques

**for (expr1 ; expr2 ; expr3) équivaut à :**

|                     |                       |
|---------------------|-----------------------|
| <b>{</b>            | <b>expr1;</b>         |
| <b>instructions</b> | <b>while(expr2)</b>   |
| <b>}</b>            | <b>{ instructions</b> |
|                     | <b>expr3;</b>         |
|                     | <b>}</b>              |

- En pratique, expr1 et expr3 contiennent souvent plusieurs initialisations ou réinitialisations, *séparées par des virgules*



## Boucle for : exemple

Calcul de  $x$  à la puissance  $n$  où  $x$  est un réel non nul et  $n$  un entier positif ou nul

```
main ()
{
 float x, puiss;
 int n, i;
 { printf (" Entrez respectivement les valeurs de x et n \n");
 scanf ("%f %d" , &x, &n);
 for (puiss =1, i=1; i<=n; i++)
 puiss*=x;
 printf (" %f à la puissance %d est égal à : %f", x,n,puiss);
 }
}
```

# L'instruction break

- L'instruction break peut être utilisée dans une boucle (for, while, ou do .. while). Elle permet d'arrêter le déroulement de la boucle et le passage à la première instruction qui la suit  
En cas de boucles imbriquées, break ne met fin qu' à la boucle la plus interne

```
#include< stdio.h>
int main(void){
 int i;
 for(i=0;i < 10;i++){
 if(i==5){
 break; // quitter la boucle
 }
 printf("i = %d \n",i);
 }
 printf("sortir de la boucle for");
 return 0;
}
```

Résultat:

**i=0**

**i=1**

**i=2**

**i=3**

**i=4**

**sortir de la boucle for**

# L'instruction continue

- L'instruction continue peut être utilisée dans une boucle (for, while, ou do .. while). Elle permet l'abandon de l'itération courante et le passage à l'itération suivante

```
• #include <stdio.h>
 int main(void){
 int i;
 for(i=0;i < 10;i++){
 if(i==5){
 continue; // passer à l'itération suivante
 }
 printf("i = %d \n",i);
 }
 printf("retour à la ligne\n");
 }
```

Résultat:

i=0  
i=1  
i=2  
i=3  
i=4  
i=6  
i=7  
i=8  
i=9

Retour à la ligne

# L'instruction continue

```
{int i;
 for(i=1;i<5;i++)
 {printf("début itération %d\n ",i);
 if(i<3) continue;
 printf(" fin itération %d\n ",i);
 }
}
```

Résultat:

}

début itération 1  
début itération 2  
début itération 3  
fin itération 3  
début itération 4  
fin itération 4



# Les Tableaux et les chaînes de caractères

# Tableaux

- Un **tableau** est une variable structurée composée d'un nombre de variables simples de même type désignées par un seul identificateur
- Ces variables simples sont appelées *éléments ou composantes* du tableau, elles sont stockées en mémoire à des emplacements contigus (l'un après l'autre)
- Le type des éléments du tableau peut être :
  - simple : char, int, float, double, ...
  - pointeur ou structure (chapitres suivants)
- On peut définir des tableaux :
  - à une dimension (tableau unidimensionnel ou vecteur)
  - à plusieurs dimensions (tableau multidimensionnel)



# Déclaration des tableaux

- La déclaration d'un tableau à une dimension s'effectue en précisant le type de ses éléments et sa dimension (le nombre de ses éléments) :
  - Syntaxe en C : **Type identificateur[dimension];**
  - Exemple : **float notes[30];**
- La déclaration d'un tableau permet de lui réserver un espace mémoire dont la taille (en octets) est égal à :  $\text{dimension} * \text{taille du type}$
- ainsi pour :
  - **short A[100];** // on réserve 200 octets ( $100 * 2 \text{ octets}$ )
  - **char mot[10];** // on réserve 10 octets ( $10 * 1 \text{ octet}$ )

# Déclaration des tableaux

- Déclarer un tableau de 5 entiers
- Un tableau de 20 caractères
- Un tableau de 100 nombres réelles en simple précision
- Un tableau de 100 nombres réelles en double précision

```
int t[5];
```

```
char a[20];
```

```
float x[100];
```

```
double y[100];
```

## Déclaration des tableaux

- En C on peut factoriser par le même type identique lors de la déclaration de plusieurs tableaux de même type. Par exemple, on peut faire les déclarations suivantes:

- `int a[10], b[20];`
- `float x[100], y[200];`

Que déclare – t-on par les instructions suivantes:

```
int a, b[100];
```

```
char c[15], d[10];
```

```
float x[10], z[20];
```

## Déclaration des tableaux

- a : un entier simple
- b : un tableau de 100 entiers
- c: un tableau de 15 caractères
- d: un tableau de 10 caractères
- x : un tableau de 10 réels simple précision
- y : un tableau de 20 réels simple précision

# Initialisation à la déclaration

- On peut initialiser les éléments d'un tableau lors de la déclaration, en indiquant la liste des valeurs respectives entre accolades. Ex:
  - `int A[5] = {1, 2, 3, 4, 5};`
  - `float B[4] = {-1.5, 3.3, 7e-2, -2.5E3};`
- Si la liste ne contient pas assez de valeurs pour toutes les composantes, les composantes restantes sont initialisées par zéro
  - Ex: `short T[10] = {1, 2, 3, 4, 5};`
- la liste ne doit pas contenir plus de valeurs que la dimension du tableau.  
Ex: `short T[3] = {1, 2, 3, 4, 5};` → Erreur
- Il est possible de ne pas indiquer la dimension explicitement lors de l'initialisation. Dans ce cas elle est égale au nombre de valeurs de la liste. Ex: `short T[] = {1, 2, 3, 4, 5};` → tableau de 5 éléments

# Initialisation à la déclaration

- Indiquer les éléments de chacun des tableaux suivants, ainsi que leur nombre:
  - `int tab1[15]={2, 3,5,7,11};`
  - `int tab2[]={0,2,4,6,8};`
  - `int tab3[100]={1};`
- Découvrir les erreurs dans les déclarations suivantes:
  - `int []={2, 3,5,7,11};`
  - `int a[3]={10,20,40,60,80};`
  - `int s[-4];`
  - `int t[4,7];`



- Indiquer les éléments de chacun des tableaux suivants, ainsi que leur nombre:
  - tab1 contient 5 éléments qui sont 2,3,5,7 et 11
  - tab2 contient 5 éléments qui sont 0,2,4,6 et 8
  - tab3 contient 100 éléments le premier élément de tab3 est 1 et les autres sont tous égaux à 0
- Découvrir les erreurs dans les déclarations suivantes:
  - Le nom du tableau n'est pas indiqué
  - Le nombre de valeurs dans la séquence d'initialisation dépasse le nombre des éléments indiqué (qui est 3)
  - Le nombre des éléments d'un tableau ne doit pas être négatif
  - Le nombre des éléments d'un tableau ne doit pas être un nombre avec virgule

# Accès aux composantes d'un tableau

- L'accès à un élément du tableau se fait au moyen de l'indice. Par exemple, **T[i]** donne la valeur de l'élément i du tableau T
- En langage C l'indice du premier élément du tableau est 0. L'indice du dernier élément est égal à la dimension-1
  - Ex:        `int T[ 5] = {9, 8, 7, 6, 5};` →    `T[0]=9, T[1]=8, T[2]=7, T[3]=6, T[4]=5`

## **Remarques:**

- on ne peut pas saisir, afficher ou traiter un tableau en entier, ainsi on ne peut pas écrire `printf(" %d",T)` ou `scanf(" %d",&T)`
- On traite les tableaux élément par élément de façon répétitive en utilisant des boucles

# Tableaux : saisie et affichage

- Saisie des éléments d'un tableau T d'entiers de taille n :

```
for(i=0;i<n;i++)
 { printf ("Entrez l'élément %d \n ",i + 1);
 scanf(" %d" , &T[i]);
 }
```

- Affichage des éléments d'un tableau T de taille n :

```
for(i=0;i<n;i++)
 printf (" %d \t",T[i]);
```

# Tableaux : exemple

- Calcul du nombre d'étudiants ayant une note supérieure à 10 :

```
main ()
{ float notes[30]; int nbre,i;
 for(i=0;i<30;i++)
 { printf ("Entrez notes[%d] \n ",i);
 scanf(" %f" , ¬es[i]);
 }
 nbre=0;
 for (i=0; i<30; i++)
 if (notes[i]>10) nbre+=1;
 printf (" le nombre de notes > à 10 est égal à : %d", nbre);
}
```

# Tableaux à plusieurs dimensions

On peut définir un tableau à n dimensions de la façon suivante:

- **Type** `Nom_du_Tableau[D1][D2]...[Dn];` où  $D_i$  est le nombre d'éléments dans la dimension  $i$
- **Exemple :** pour stocker les notes de 20 étudiants en 5 modules dans deux examens, on peut déclarer un tableau :

**`float notes[20][5][2];`**

(`notes[i][j][k]` est la note de l'examen  $k$  dans le module  $j$  pour l'étudiant  $i$ )



# Tableaux à deux dimensions (Matrices)

- Syntaxe : `Type nom_du_Tableau[nombre_ligne][nombre_colonne];`
- Ex: **`short A[2][3];`** On peut représenter le tableau A de la manière suivante :

|                      |                      |                      |
|----------------------|----------------------|----------------------|
| <code>A[0][0]</code> | <code>A[0][1]</code> | <code>A[0][2]</code> |
| <code>A[1][0]</code> | <code>A[1][1]</code> | <code>A[1][2]</code> |

- Un tableau à deux dimensions `A[n][m]` est à interpréter comme un tableau unidimensionnel de dimension `n` dont chaque composante `A[i]` est un tableau unidimensionnel de dimension `m`.
- Un tableau à deux dimensions `A[n][m]` contient  $n * m$  composantes. Ainsi lors de la déclaration, on lui réserve un espace mémoire dont la taille (en octets) est égal à :  $n * m * \text{taille du type}$



# Initialisation à la déclaration d'une Matrice

- L'initialisation lors de la déclaration se fait en indiquant la liste des valeurs respectives entre accolades ligne par ligne
- Exemple :
  - `float A[3][4] = {{-1.5, 2.1, 3.4, 0}, {8e-3, 7e-5, 1, 2.7 }, {3.1, 0, 2.5E4, -1.3E2}};`  
  
`A[0][0]=-1.5 , A[0][1]=2.1, A[0][2]=3.4, A[0][3]=0`  
`A[1][0]=8e-3 , A[1][1]=7e-5, A[1][2]=1, A[1][3]=2.7`  
`A[2][0]=3.1 , A[2][1]=0, A[2][2]=2.5E4, A[2][3]=-1.3E2`
- On peut ne pas indiquer toutes les valeurs: Les composantes manquantes seront initialisées par zéro

# Matrices : saisie et affichage

- Saisie des éléments d'une matrice d'entiers  $A[n][m]$  :

```
for(i=0;i<n;i++)
 for(j=0;j<m;j++)
 { printf ("Entrez la valeur de A[%d][%d] \n ",i,j); scanf(" %d" , &A[i][j]);
 }
```

- Affichage des éléments d'une matrice d'entiers  $A[n][m]$  :

```
for(i=0;i<n;i++)
 { for(j=0;j<m;j++)
 printf (" %d \t",A[i][j]);
 printf("\n");
 }
```

# Représentation d'un tableau en mémoire

- La déclaration d'un tableau provoque la réservation automatique par le compilateur d'une zone contiguë de la mémoire.
- La mémoire est une succession de cases mémoires. Chaque case est une suite de 8 bits (1 octet), identifiée par un numéro appelé **adresse**. (on peut voir la mémoire comme une armoire constituée de tiroirs numérotés. Un numéro de tiroir correspond à une adresse)
- Les adresses sont souvent exprimées en hexadécimal pour une écriture plus compacte et proche de la représentation binaire de l'adresse. Le nombre de bits d'adressage dépend des machines.
- En C, l'opérateur **&** désigne **adresse de**. Ainsi, `printf(" adresse de a=%x ", &a)` affiche l'adresse de la variable a en hexadécimal

# Représentation d'un tableau à une dimension en mémoire

- En C, le nom d'un tableau est le représentant de l'adresse du premier élément du tableau (pour un tableau T:  **$T = \&T[0]$**  )
- Les composantes du tableau étant stockées en mémoire à des emplacements contigus, les adresses des autres composantes sont calculées (automatiquement) relativement à cette adresse :

$$\&T[i] = \&T[0] + \text{sizeof}(\text{type}) * i$$

- Exemple :  **$\text{short } T[5] = \{100, 200, 300, 400, 500\};$**   
et supposons que  **$T = \&T[0] = 1E06$**
- On peut afficher et vérifier les adresses du tableau:

```
for(i=0;i<5;i++)
printf("adresse de T[%d]= %x\n",i,&T[i]);
```

|          |     |
|----------|-----|
| 1E05     |     |
| T → 1E06 | 100 |
| 1E08     | 200 |
| 1E0A     | 300 |
| 1E0C     | 400 |
| 1E0E     | 500 |
| 1E0F     |     |

# Représentation d'un tableau à deux dimensions en mémoire

- Les éléments d'un tableau sont stockés en mémoire à des emplacements contigus ligne après ligne
- Comme pour les tableaux unidimensionnels, le nom d'un tableau A à deux dimensions est le représentant de l'adresse du premier élément :  **$A = \&A[0][0]$**
- Rappelons qu'une matrice  $A[n][m]$  est à interpréter comme un tableau de dimension n dont chaque composante  $A[i]$  est un tableau de dimension m.  
 **$A[i]$  et  $\&A[i][0]$  représentent l'adresse du 1<sup>er</sup> élément de la ligne i (pour i de 0 à n-1)**
- Exemple :  **$\text{char } A[3][4];$**   $A = \&A[0][0] = 0118$





# Chaînes de caractères

- Il n'existe pas de type spécial chaîne ou string en C. Une chaîne de caractères est traitée comme un tableau de caractères
- Une chaîne de caractères en C est caractérisée par le fait que le dernier élément vaut le caractère ‘\0’, ceci permet de détecter la fin de la chaîne
- Il existe plusieurs fonctions prédéfinies pour le traitement des chaînes de caractères (ou tableaux de caractères )



# Déclaration

- Syntaxe : **char <NomVariable> [<Longueur>];** //tableau de caractères

Exemple : **char NOM [15];**

- Pour une chaîne de N caractères, on a besoin de N+1 octets en mémoire (le dernier octet est réservé pour le caractère ‘\0’)
- Le nom d’une chaîne de caractères est le représentant de l’adresse du 1<sup>er</sup> caractère de la chaîne

## Exemple

```
int main() {
 char ligne[10]; // Déclaration du tableau pour 9 lettres + le
 caractère nul '\0'
 int i;

 for(i = 0; i < 9; i++) {
 ligne[i] = 'a' + i; // Remplit le tableau avec les lettres de 'a' à 'i'
 }
 ligne[9] = '\0'; // Ajoute le caractère nul pour terminer la chaîne

 printf("%s\n", ligne); // Affiche la chaîne
}
```

# Initialisation

- On peut initialiser une chaîne de caractères à la définition :
  - comme un tableau, par exemple : `char ch[ ] = {'e','c','o','l','e','\0'}`
  - par une chaîne constante, par exemple : `char ch[ ] = "école"`
- On peut préciser le nombre d'octets à réserver à condition que celui-ci soit supérieur ou égal à la longueur de la chaîne d'initialisation
  - `char ch[ 6] = "ecole"` est valide
  - `char ch[ 4] = "ecole"` ou `char ch[ 5] = "ecole"` provoque une erreur

# Traitement des chaînes de caractères

- Le langage C dispose d'un ensemble de bibliothèques qui contiennent des fonctions spéciales pour le traitement de chaînes de caractères
- Les principales bibliothèques sont :
  - La bibliothèque **<stdio.h>**
  - La bibliothèque **<string.h>**
  - La bibliothèque **<stdlib.h>**
- Nous verrons les fonctions les plus utilisées de ces bibliothèques

## Fonctions de la bibliothèque <stdio.h>

- **printf( )** : permet d'afficher une chaîne de caractères en utilisant le spécificateur de format %s.

Exemple : **char ch[ ]= " Bonsoir " ;**  
**printf(" %s ", ch)**

- **puts( <chaîne> )** : affiche la chaîne de caractères désignée par <Chaîne> et provoque un retour à la ligne.

Exemple : **puts(ch);** /\*équivalente à printf("%s\n ", ch);\*/

# Fonctions de la bibliothèque <stdio.h>

```
char S[8]= " bonjour " ;
```

- `printf(" %c " , S[0]);`      `// affiche b`
- `putchar(S[0]) ;`      `// affiche b`
- `printf(" %s " , S);`      `// affiche bonjour`
- `Puts(S);`      `// affiche bonjour`



# Fonctions de la bibliothèque <stdio.h>

- **scanf( )** : permet de saisir une chaîne de caractères en utilisant le spécificateur de format %s.

Exemple : **char Nom[15];**

**printf("entrez votre nom");**

**scanf(" %s ", Nom);**

**Remarque :** le nom d'une chaîne de caractères est le représentant de l'adresse du premier caractère de la chaîne, il ne doit pas être précédé de &

- **gets( <chaîne> )** : lit la chaîne de caractères désignée par <Chaîne>

Exemple : **char phrase[100];**

**printf("entrez une phrase");**

**gets(phrase);**

## Fonctions de la bibliothèque <string.h>

- **strlen(ch)**: fournit la longueur de la chaîne sans compter le '\0' final

Exemple : **char s[ ]= " Test";**

**printf("%d",strlen(s));** //affiche 4

- **strcat(ch1, ch2)** : ajoute ch2 à la fin de ch1. Le caractère '\0' de ch1 est écrasé par le 1<sup>er</sup> caractère de ch2

Exemple : **char ch1[20]=" Bonne ", ch2=" chance ";**

**strcat(ch1, ch2) ;**

**printf(" %s", ch1);** // affiche Bonnechance

# Fonctions de la bibliothèque <string.h>

- **strcmp(ch1, ch2):** compare ch1 et ch2 lexicographiquement et retourne une valeur :
  - nul si ch1 et ch2 sont identiques
  - négative si ch1 précède ch2
  - positive si ch1 suit ch2
- **strcpy(ch1, ch2) :** copie ch2 dans ch1 y compris le caractère ‘\0’

Exemple :     **char ch[10];**  
                  **strcpy(ch, " Bonjour ");**  
                  **puts(ch);** // affiche Bonjour

- **strchr(char \*s, char c) :** recherche la 1<sup>ère</sup> occurrence du caractère c dans la chaîne s et retourne un pointeur sur cette 1<sup>ère</sup> occurrence si c’est un caractère de s, sinon le pointeur NULL

# Fonctions de la bibliothèque <stdlib.h>

<stdlib> contient des fonctions pour la conversion de nombres en chaînes de caractères et vice-versa.

- **atoi(ch)**: retourne la valeur numérique représentée par ch comme **int**
- **atof(ch)**: retourne la valeur numérique représentée par ch comme **float**  
(si aucun caractère n'est valide, ces fonctions retournent 0) Exemple :  
**int x, float y;**  
**char \*s= " 123 ", ch[]= " 4.56 "; x=atoi(s); y=atof(ch);**  
**// x=123 et y=4.56**
- **itoa(int n, char \* ch, int b)** : convertit l'entier n en une chaîne de caractères qui sera attribué à ch. La conversion se fait en base b  
Exemple : **char ch[30]; int p=18;**  
**itoa(p, ch, 2); // ch= " 10010 ";**



# Les Pointeurs



- Toute variable manipulée dans un programme est stockée quelque part en **mémoire centrale**.
- Cette mémoire est constituée d'octets qui sont identifiés de manière univoque par un numéro qu'on appelle **adresse**.
- Pour retrouver **une variable**, il suffit donc de connaître **l'adresse de l'octet ou elle est stockée**.
- Pour des raisons évidentes de lisibilité, on désigne souvent les variables par des **identificateurs**, et non par leur adresse.
- C'est **le compilateur** qui fait alors le lien entre l'identificateur d'une variable et son adresse en mémoire.
- Toutefois, il est parfois très pratique de manipuler directement une variable par son adresse.



- On appelle **Lvalue** (left value) tout objet pouvant être placé à gauche d'un opérateur d'affectation.
- Une **Lvalue** est caractérisée par:
  - son adresse**, c'est-à-dire l'adresse-mémoire à partir de laquelle l'objet est stocké;
  - sa valeur**, c'est-à-dire ce qui est stocké à cette adresse.

Dans l'exemple,

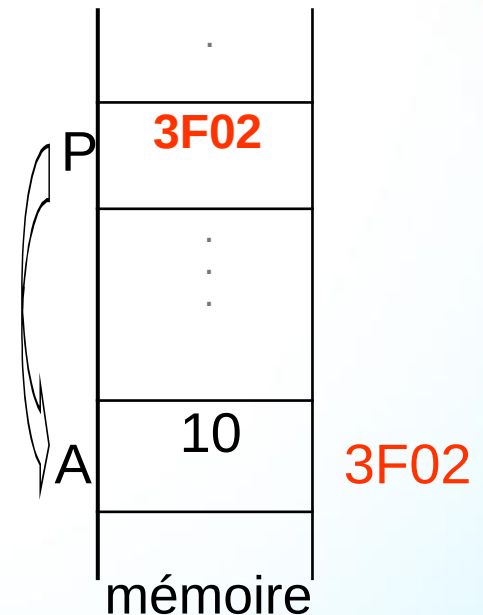
- `int i, j;`  
`i = 3;`  
`j = i;`

| objet | adresse    | valeur |
|-------|------------|--------|
| i     | 4831836000 | 3      |
| j     | 4831836004 | 3      |

- Deux variables différentes ont des adresses différentes. L'affectation `j = i;` n'opère que sur les valeurs des variables.

# Pointeurs : définition

- Un pointeur est une variable spéciale qui peut contenir l'adresse d'une autre variable.
- Exemple : Soit A une variable contenant la valeur 10 et P un pointeur qui contient l'adresse de A (*on dit que P pointe sur A*) .
- Remarques :
  - Le nom d'une variable permet d'accéder *directement* à sa valeur (**adressage direct**).
  - Un pointeur qui contient l'adresse de la variable, permet d'accéder *indirectement* à sa valeur (**adressage indirect**).
  - Le nom d'une variable est lié à la même adresse, alors qu'un pointeur peut pointer sur différentes adresses



# Intérêts des pointeurs

- Les pointeurs présentent de nombreux avantages :
  - Ils sont indispensables pour permettre le passage par référence pour les paramètres des fonctions
  - Ils permettent de créer des structures de données (listes et arbres) dont le nombre d'éléments peut évoluer dynamiquement. Ces structures sont très utilisées en programmation.
  - Ils permettent d'écrire des programmes plus compacts et efficaces

# Déclaration d'un pointeur

- En C, chaque pointeur est limité à un type de donnée (même si la valeur d'un pointeur, qui est une adresse, est toujours un entier).
- Le type d'un pointeur dépend du type de la variable pointée. Ceci est important pour connaître la taille de la valeur pointée.
- On déclare un pointeur par l'instruction : **type \*nom-du-pointeur ;**
  - type est le type de la variable pointée
  - \* est l'opérateur qui indiquera au compilateur que c'est un pointeur (désigne en fait le contenu de l'adresse)

# Déclaration d'un pointeur

- Exemple :

```
int *pi; //pi est un pointeur vers une variable de type int
float *pf; //pf est un pointeur vers une variable de type float
char *b; /* un pointeur b vers un caractère*/
double *y; /* un pointeur a vers un réel double*/
```

- Rq: la valeur d'un pointeur donne l'adresse du premier octet parmi les n octets où la variable est stockée



## Déclaration d'un pointeur

- Dans la déclaration d'un pointeur, le symbole **\*** est associé au nom de la variable pointeur et non pas au type
- Ainsi la déclaration:

**int \*a,b;**

Ne déclare pas deux pointeurs a et b, mais un pointeur a vers un int et une variable simple b de type int.

- Pour déclarer deux pointeurs a et b dans la même ligne, il faut écrire:

**int\*a,\*b;**



# Opérateurs de manipulation des pointeurs

- Lors du travail avec des pointeurs, nous utilisons :
  - un **opérateur 'adresse de':** **&** pour obtenir l'adresse d'une variable
  - un **opérateur 'contenu de':** **\*** pour accéder au contenu d'une adresse
- Exemple :
  - **int \* p;** //on déclare un pointeur vers une variable de type int
  - **int i=10, j=30;** // deux variables de type int
  - **p=&i;** // on met dans p, l'adresse de i (p pointe sur i)
  - **printf("\*p = %d \n",\*p);** //affiche : \*p = 10
  - **\*p=20;** // met la valeur 20 dans la case mémoire pointée par p (i vaut 20 après cette instruction)
  - **p=&j;** // p pointe sur j
  - **i=\*p;** // on affecte le contenu de p à i (i vaut 30 après cette instruction)

# Opérateurs de manipulation des pointeurs

- Considérons un pointeur  $p$  qui pointe vers une variable  $a$  de type  $T$ .
- L'expression  $*p$  est équivalente à  $a$ . Donc, on peut manipuler la variable  $a$  :
  - soit directement par son nom
  - soit indirectement par  $*p$ .
- En fait  $*p$  est considéré comme une variable de type  $T$  et tout ce qui applicable sur  $a$ , on peut l'appliquer sur  $*p$

# Opérateurs de manipulation des pointeurs

- Exemple :

Déclarer une variable de type int et l'initialiser par la valeur 5, puis déclarer un pointeur de type int et l'initialiser par l'adresse de la variable déclarée.

```
int a=5;
int*p=&a;
```

Modifier la valeur de a à 9 en utilisant les deux méthodes citées dans la définition.

```
a=9;/*modification directe de la valeur de a*/
p=9;/ modification indirecte à l'aide d'un pointeur*/
```

# Opérateurs de manipulation des pointeurs

- Exemple2 : `float a, *p;`  
`p=&a;`  
`printf("Entrez une valeur : \n");`  
`scanf("%f ",p);` //supposons qu'on saisit la valeur 1.5  
`printf("Adresse de a= %x, contenu de a= %f\n" ,`  
`p,*p);`  
`*p+=0.5;`  
`printf ("a= %f\n" , a);` //affiche a=2.0
- **Remarque** : si un pointeur P pointe sur une variable X, alors \*P peut être utilisé partout où on peut écrire X
  - X+=2 équivaut à \*P+=2
  - ++X équivaut à ++ \*P
  - X++ équivaut à (\*P)++ // les parenthèses ici sont obligatoires car l'associativité des opérateurs unaires \* et ++ est de droite à gauche

# Exercice

Déterminer la valeur de la variable a après l'exécution de chacune des instructions suivantes:

```
int a=4;
```

```
int *p=&a;
```

```
(*p)++;
```

```
*p=(*p)*(*p);
```

## Solution:

- ✓ a est une variable de type int initialisée à 4.
- ✓ p est un pointeur vers a. Donc \*p vaut 4.
- ✓ (\*p)++ permet incrémenter l'objet pointé par p, à savoir a. La valeur de a vaut maintenant 5.
- ✓ \*p=(\*p)\*(\*p), permet de calculer le carré de a et de l'affecter à a. Donc la variable a vaut 25.



# Initialisation d'un pointeur

- A la déclaration d'un pointeur p, on ne sait pas sur quel zone mémoire il pointe. Ceci peut générer des problèmes :
  - `int *p;`  
`*p = 10;` //provoque un problème mémoire car le pointeur p n'a pas été initialisé
- **Conseil** : Toute utilisation d'un pointeur doit être précédée par une initialisation.
- On peut initialiser un pointeur en lui affectant :
  - l'adresse d'une variable (Ex: `int a, *p1; p1=&a;` )
  - un autre pointeur déjà initialisé (Ex: `int *p2; p2=p1;`)
  - la valeur 0 désignée par le symbole NULL, défini dans `<stddef.h>`.  
Ex: `int *p; p=0;` ou `p=NULL;` (on dit que p pointe 'nulle part': aucune adresse mémoire ne lui est associé)
- Rq: un pointeur peut aussi être initialisé par une allocation dynamique (voir fin du chapitre)

# Affectation de pointeurs

- L'affectation d'un pointeur à un autre est possible si les deux pointeurs sont de même type T, alors **p=q**; permet d'affecter le pointeur q au pointeur p.
- Cela a pour effet de faire pointer p et q vers la même variable.
- Exemple:  
Déclarer une variable de type int et l'initialisée par la valeur 7, puis déclarer deux pointeurs de type int qui vont pointer vers la variable déclarée.

```
int a=7;
int*p=&a;
int*q=p;
```

# Opérations arithmétiques avec les pointeurs

- La valeur d'un pointeur étant un entier, certaines opérations arithmétiques sont possibles : ajouter ou soustraire un entier à un pointeur ou faire la différence de deux pointeurs
- Pour un entier  $i$  et des pointeurs  $p$ ,  $p1$  et  $p2$  sur une variable de type  $T$ 
  - **$p+i$  (resp  $p-i$ )** : désigne un pointeur sur une variable de type  $T$ . Sa valeur est égale à celle de  $p$  incrémentée (resp décrémentée) de  $i*\text{sizeof}(T)$ .
  - **$p1-p2$**  : Le résultat est un entier dont la valeur est égale à (différence des adresses)/ $\text{sizeof}(T)$ .
- Remarque:
  - on peut également utiliser les opérateurs  $++$  et  $--$  avec les pointeurs
  - la somme de deux pointeurs n'est pas autorisée

# Opérations arithmétiques avec les pointeurs

- Exemples:

1. Soit `a` une variable de type `int` et `p` est un pointeur vers `a`. Sachant que le type est codé sur 4 octets et que l'adresse de la variable `a` est 1030:

- quel est le contenu du pointeur `p+3`.
- quel est le contenu du pointeur `p-3`.

2. La variable `a` de type `int` est rangée en mémoire à l'adresse 1000.

Quel est le contenu de chacun des pointeurs suivants:

1.

```
int *p=&a;
int*q=p++;
int*r=++q;
```

2.

```
int *p=&a;
int*q=p--;
int*r=--q;
```

# Opérations arithmétiques avec les pointeurs

1.

Le pointeur  $p$  pointe vers  $a$ , donc le contenu de  $p$  est l'adresse de  $a$ , c'est-à-dire 1030.

- Le pointeur  $p+3$  pointe vers l'entier qui se trouve 3 fois après celui que contient la variable  $a$ . Donc le contenu du pointeur  $p+3$  est  $1030+3*4=1042$ .
- Le pointeur  $p-3$  pointe vers l'entier qui se trouve 3 fois avant celui que contient la variable  $a$ . Donc le contenu du pointeur  $p-3$  est  $1030-3*4=1018$ .



# Opérations arithmétiques avec les pointeurs

## 2.

Le pointeur p pointe vers a, donc le contenu de p est l'adresse de p, c'est-à-dire 1000.

- $q=p++$ ; est équivalente à  $q=p$ ;  $p=p+1$ ; donc le contenu du pointeur q est 1000, mais le pointeur p est incrémenté et par suite, le contenu de est 1004.
- $r=++q$ ; est équivalente à  $q=q+1$ ;  $r=q$ ; Donc le contenu de q est 1004, et celui de r est 1004
- $q=p--$ ; est équivalente à  $q=p$ ;  $p=p-1$ ; Donc le contenu du pointeur q est 1000, mais le pointeur p est décrémenté et par la suite, le contenu de p est 996.
- $r=--q$ ; est équivalente à  $q=q-1$ ;  $r=q$ ; Donc le contenu de q est 996, et celui de r est 996.



## Exercice 1

```
main(){
```

```
short A, B, *P; /*supposons que ces variables occupent
la mémoire à partir de l'adresse 01A0 */
```

```
A = 10;
```

```
B = 50;
```

```
P = &A ;
```

```
B = *P;
```

```
*P = 20;
```

```
P = &B;
```

```
*P += 15;
```

```
}
```

Donnez les valeurs de A, B, P après chaque instruction

## Exercice 2

```
main(){
```

```
float a , *p; /*supposons que ces variables sont
représentées en mémoire à partir de l'adresse 01BE*/
```

```
p = &a;
```

```
printf("Entrer une valeur réelle:");
```

```
scanf("%f",p); // on saisie la valeur 1.4
```

```
printf("\nAdresse de a = %x Contenu de a = %f",p,*p);
```

```
*p += 0.4;
```

```
printf("\na = %f ", a);
```

```
}
```

Donnez les valeurs de a et p après chaque instruction

## Exercise 3

```
int X=4,Y ;
```

```
int *p,*q ;
```

```
p=&X ;
```

```
printf("%d ",*p) ;
```

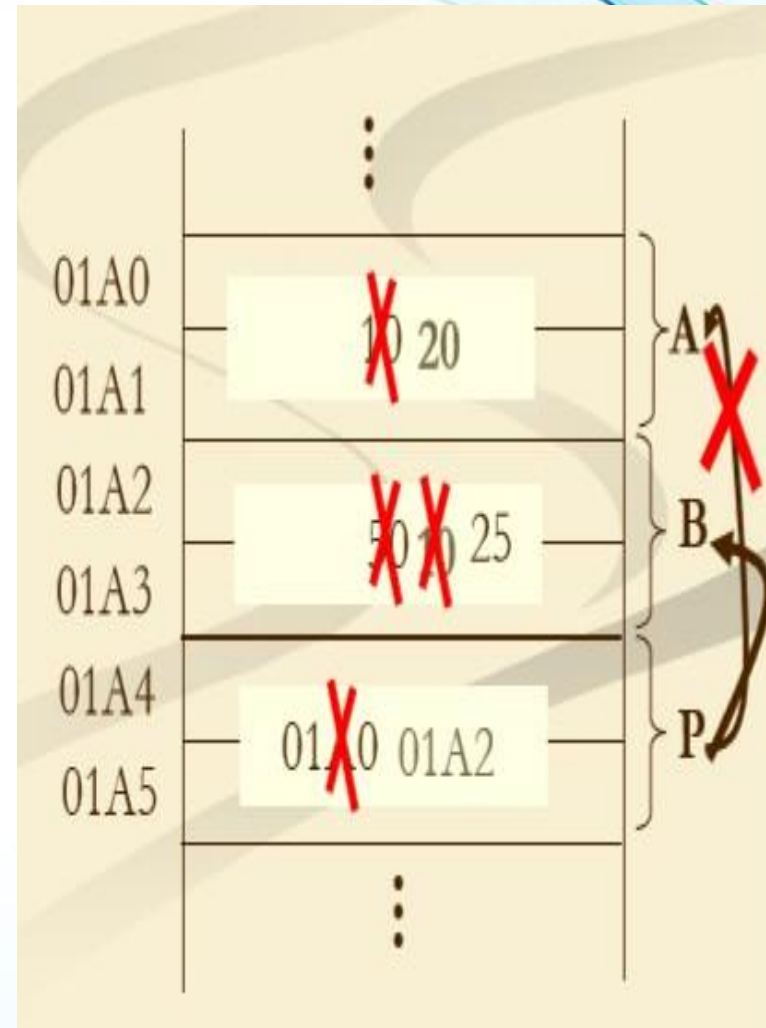
```
Y=*p-1 ;
```

```
*p+=1 ;
```

```
(*p)++ ;
```

# Solution 1

```
main(){
short A, B, *P; /*supposons que ces
variables occupent la mémoire à
partir de l'adresse 01A0 */
A = 10;
B = 50;
P = &A ; // se lit mettre dans P
l'adresse de A
B = *P; /* mettre dans B le contenu
de l'emplacement mémoire
pointé par P */
*P = 20; /*mettre la valeur 20 dans
l'emplacement
mémoire pointé par P */
P = &B; // P pointe sur B
*P += 15; // additionner 15 au
contenu de l'adresse sur
laquelle pointe P
}
```



# Solution 2

```
main(){
```

```
float a , *p; /*supposons que
ces variables sont
représentées en mémoire à
partir de l'adresse 01BE*/
```

```
p = &a;
```

```
printf("Entrer une valeur
réelle:");
```

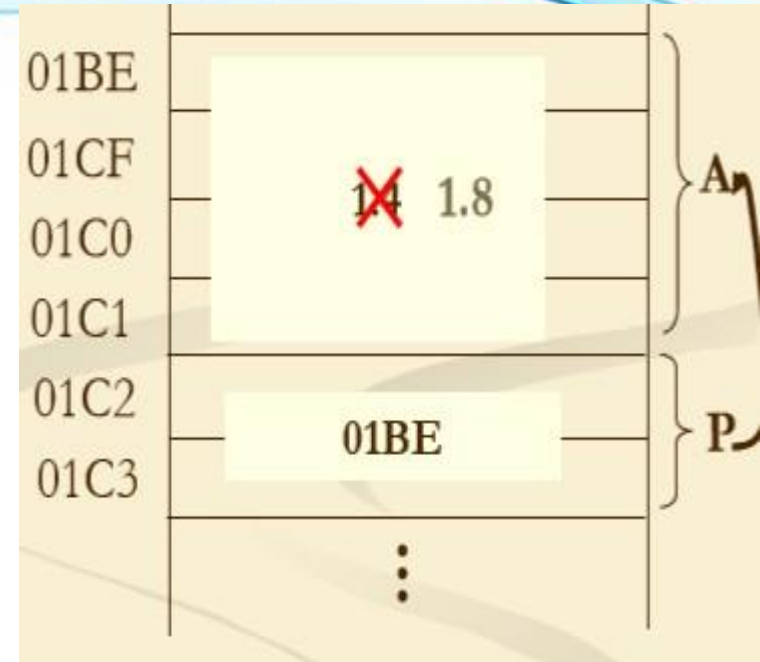
```
scanf("%f",p); // on saisie la
valeur 1.4
```

```
printf("\nAdresse de a = %x
Contenu de a = %f",p,*p);
```

```
*p += 0.4;
```

```
printf("\na = %f ", a);
```

```
}
```



Affichage sur Ecran

Entrer une valeur réelle : 1.4

Adresse de a : 01BE Contenu de a = 1.4

a = 1.8



## Solution 3

```
int X=4,Y ;
int *p,*q ;
p=&X ; /* p contient l'adresse de X*/
printf("%d",*p) ; /* affiche le contenu de la variable
pointée par le pointeur p (c.-à-d.
la valeur de x).*/
Y=*p-1 ; /* Y vaut 3 */
p+=1 ; / incrémente X de 1*/
(*p)++ ; /* incrémente aussi de 1 la variable pointée par p
, X vaut 6*/
/* N.B : les parenthèses sont importantes. Car *p++
incrémente le pointeur p (l'adresse) et non pas la valeur
de X. */
```



# Pointeurs et tableaux

- Comme on l'a déjà vu au chapitre « Tableaux », le nom d'un tableau T représente l'adresse de son premier élément ( $T = \&T[0]$ ). Avec le formalisme pointeur, on peut dire que T est un **pointeur constant** sur le premier élément du tableau.
- En déclarant un tableau T et un pointeur P du même type, l'instruction  $P = T$  fait pointer P sur le premier élément de T ( $P = \&T[0]$ ) et crée une liaison entre P et le tableau T.
- A partir de là, on peut manipuler le tableau T en utilisant P, en effet:

P    pointe sur T[0]    et    \*P    désigne T[0]

P+1    pointe sur T[1]    et    \*(P+1) désigne T[1]

....

P+i    pointe sur T[i]    et    \*(P+i) désigne T[i]

# Pointeurs et tableaux

- Si on déclare un tableau statique et un pointeur ainsi:

```
int tab[10];
int *p;
```

alors l'affectation : **p=&tab[0]** est équivalente à **p=tab** et **\*p** désigne **tab[0]**.

- Les crochets sont une simplification d'écriture: **tab[i] ⇔ \*(tab+i)**
- Il existe tout de même une différence entre pointeur et tableau: un nom de tableau n'est pas une variable, on ne peut donc rien affecter au nom de tableau **tab=p** (**incorrecte**) , contrairement au pointeur alors **p=tab**

# Pointeurs et tableaux

- Exemple: `short x, A[7]={5,0,9,2,1,3,8};`  
`short *P;`  
`P=A;`  
`x=*(P+5);`
- Le compilateur obtient l'adresse  $P+5$  en ajoutant  $5 * \text{sizeof}(\text{short}) = 10$  octets à l'adresse dans  $P$
- D'autre part, les composantes du tableau sont stockées à des emplacements contigus et  $\&A[5] = \&A[0] + \text{sizeof}(\text{short}) * 5 = A + 10$
- Ainsi,  $x$  est égale à la valeur de  $A[5]$  ( $x = A[5]$ )

# Pointeurs et tableaux

- Placer la valeur 1 dans chacun des 10 éléments de notre tableau tab

```
int i ;
for (i=0 ; i<10 ; i++)
 * (tab+i) = 1 ;
```

```
int i ;
int * p :
for (p=tab, i=0 ; i<10 ; i++, p++)
 * p = 1 ;
```

# Pointeurs et tableaux

- Addition et soustraction d'un nombre entier ;

Si P pointe sur l'élément `tab[i]` d'un tableau, alors

`P+n`            pointe sur `tab[i+n]`

`P-n`            pointe sur `tab[i-n]`

- Incrémentation et décrémentation d'un pointeur :

Si P pointe sur l'élément `tab[i]` d'un tableau, alors après l'instruction

`P++;`            P pointe sur `tab[i+1]`

`P+=n;`           P pointe sur `tab[i+n]`

`P--;`            P pointe sur `tab[i-1]`

`P-=n;`           P pointe sur `tab[i-n]`



# Pointeurs et tableaux: saisie et affichage d'un tableau

Version 1:

```
main()
{ float T[100] , *pt;
 int i,n;
 do {printf("Entrez n \n ");
 scanf(" %d" ,&n);
 }while(n<0 ||n>100);

 pt=T;
 for(i=0;i<n;i++)
 { printf ("Entrez T[%d] \n ", i+1);
 scanf(" %f" , pt+i);
 }

 for(i=0;i<n;i++)
 printf (" %f \t",*(pt+i));
 }
```

Version 2: sans utiliser i

```
main()
{ float T[100] , *pt;
 int n;
 do {printf("Entrez n \n ");
 scanf(" %d" ,&n);
 }while(n<0 ||n>100);

 for(pt=T;pt<T+n;pt++)
 { printf ("Entrez T[%d] \n ",pt-T);
 scanf(" %f" , pt);
 }

 for(pt=T;pt<T+n;pt++)
 printf (" %f \t",*pt);
 }
```



# Pointeurs et tableaux à deux dimensions

- Le nom d'un tableau A à deux dimensions est un pointeur constant sur le premier élément du tableau c-à-d A[0][0].
- En déclarant un tableau A[n][m] et un pointeur P du même type, on peut manipuler le tableau A en utilisant le pointeur P en faisant pointer P sur le premier élément de A (P=&A[0][0]), Ainsi :

|           |                    |               |                 |
|-----------|--------------------|---------------|-----------------|
| P         | pointe sur A[0][0] | et *P         | désigne A[0][0] |
| P+1       | pointe sur A[0][1] | et *(P+1)     | désigne A[0][1] |
| ....      |                    |               |                 |
| P+M       | pointe sur A[1][0] | et *(P+M)     | désigne A[1][0] |
| ....      |                    |               |                 |
| • P+i*M   | pointe sur A[i][0] | et *(P+i*M)   | désigne A[i][0] |
|           | ....               |               |                 |
| • P+i*M+j |                    | et *(P+i*M+j) | désigne A[i][j] |

# Pointeurs : saisie et affichage d'une matrice

```
#define N 10
#define M 20
main()
{ int i, j, A[N][M], *pt;
 pt=&A[0][0];

 for(i=0;i<N;i++)
 for(j=0;j<M;j++)
 { printf ("Entrez A[%d][%d]\n ",i,j);
 scanf(" %d" , pt+i*M+j);
 }

 for(i=0;i<N;i++)
 { for(j=0;j<M;j++)
 printf (" %d \t",*(pt+i*M+j));
 printf ("\n");
 }
}
```

# Pointeurs et tableaux : remarques

En C, on peut définir :

- **Un tableau de pointeurs :**

Ex : `int *T[10];` //déclaration d'un tableau de 10 pointeurs d'entiers

```
int a = 5, b = 10, c = 15;
```

```
int *T[10]; // tableau de 10 pointeurs d'entiers
```

```
// Assignment de certains éléments du tableau à des adresses de variables
```

```
T[0] = &a;
```

```
T[1] = &b;
```

```
T[2] = &c;
```

```
// Accès aux valeurs pointées par les pointeurs
```

```
printf("%d\n", *T[0]); // Affiche 5
```

```
printf("%d\n", *T[1]); // Affiche 10
```

```
printf("%d\n", *T[2]); // Affiche 15
```

# Pointeurs et tableaux : remarques

En C, on peut définir :

- **Un pointeur de tableaux :**

Ex : **int (\*pt)[20];** //déclaration d'un pointeur sur un tableau de 20 éléments

```
int tableau[20];
int (*pt)[20] = &tableau; // pt pointe vers le tableau de 20 entiers

// Accéder aux éléments du tableau via le pointeur pt :
for (int i = 0; i < 20; i++) {
 printf("%d ", (*pt)[i]);
}
```

## pointeurs et chaines de caractères

- Les chaînes de caractères sont comme les tableaux et possèdent une adresse mémoire. L'affectation suivante:

`char *p="bonjour";` Equivalent à: `char *p  
p="bonjour";`

- Signifie que l'on définit une variable pointeur 'p' de type char\* à laquelle on affecte, comme valeur initiale, non pas la chaîne "bonjour", mais l'adresse de cette dernière
- Les instructions suivantes montrent comment afficher toute la chaîne de caractères en partant de 'p':

```
Char *p= "bonjour";
for(p=p;*p!='\0';p++)
printf("/c",*p);
```

**Affiche: bonjour**



# Pointeurs et tableaux : remarques

On peut aussi créer des tableaux de pointeurs pour manipuler plusieurs chaînes de caractères. Par ex ici jours est un tableau de pointeurs de caractères. Chaque élément du tableau pointe vers une chaîne constante (comme "Lundi").

```
#include <stdio.h>

int main() {
 const char *jours[] = {
 "Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi", "Samedi", "Dimanche"
 };

 for (int i = 0; i < 7; i++) {
 printf("%s\n", jours[i]);
 }

 return 0;
}
```



# Allocation dynamique de mémoire

- Quand on déclare une variable dans un programme, on lui réserve implicitement un certain nombre d'octets en mémoire. Ce nombre est connu avant l'exécution du programme
- Or, il arrive souvent qu'on ne connaît pas la taille des données au moment de la programmation. On réserve alors l'espace maximal prévisible, ce qui conduit à un gaspillage de la mémoire
- Il serait souhaitable d'allouer la mémoire en fonction des données à saisir (par exemple la dimension d'un tableau)
- Il faut donc un moyen pour allouer la mémoire lors de l'exécution du programme : c'est l'allocation dynamique de mémoire

## La fonction malloc

- La fonction **malloc** de la bibliothèque `<stdlib>` permet de localiser et de réserver de la mémoire, sa syntaxe est : **malloc(N)**
- Cette fonction retourne un pointeur de type `char *` pointant vers le premier octet d'une zone mémoire libre de N octets ou le pointeur `NULL` s'il n'y a pas assez de mémoire libre à allouer.
- Exemple : Si on veut réserver la mémoire pour un texte de 1000 caractères, on peut déclarer un pointeur `pt` sur **`char *pt`**.
  - L'instruction: **`pt = (char *) malloc(1000);`** fournit l'adresse d'un bloc de 1000 octets libres et l'affecte à `pt`. S'il n'y a pas assez de mémoire, `pt` obtient la valeur zéro (`NULL`).
- Remarque : Il existe d'autres fonctions d'allocation dynamique de mémoire dans la bibliothèque `<stdlib>`

## La fonction malloc et free

- Si on veut réserver de la mémoire pour des données qui ne sont pas de type char, il faut convertir le type de la sortie de la fonction malloc à l'aide d'un cast.
- Exemple : on peut réserver la mémoire pour 2 variables contiguës de type int avec l'instruction : **p = (int\*)malloc(2 \* sizeof(int));** où p est un pointeur sur **int** (int \*p).
- Si on n'a plus besoin d'un bloc de mémoire réservé par **malloc**, alors on peut le libérer à l'aide de la fonction **free**, dont la syntaxe est : **free(pointeur);**
- Si on ne libère pas explicitement la mémoire à l'aide de **free**, alors elle est libérée automatiquement à la fin du programme.

# La fonction malloc et free

## Saisie et affichage d'un tableau

```
#include<stdio.h>
#include<stdlib.h>
main()
{ float *pt;
 int i,n;
 printf("Entrez la taille du tableau \n");
 scanf(" %d" ,&n);

 pt=(float*) malloc(n*sizeof(float));
 if (pt==Null)
 {
 printf(" pas assez de mémoire \n");
 system(" pause ");
 }
}
```

```
printf(" Saisie du tableau \n ");
for(i=0;i<n;i++)
{ printf ("Élément %d ? \n ",i+1);
 scanf(" %f" , pt+i);
}

printf(" Affichage du tableau \n ");
for(i=0;i<n;i++)
 printf (" %f \t",*(pt+i));
free(pt);
}
```

# La fonction calloc

- La fonction **calloc(K,N)** retourne un pointeur vers une zone mémoire formée d'un tableau de K éléments de taille identique égale à N octets .
- Elle retourne la valeur Null en cas d'échec.
- La zone mémoire réservé par calloc est constituée de K x N octets, tous initialisés à la valeur 0.
- Le tableau renvoyé par calloc est de type char\*.
- Exemple:  
**int n=100;**  
**long \*tab=(long\*) calloc (n, sizeof(long)); /\* n fois 0 \*/**



## Fonction realloc

- La fonction **realloc(p,N)** permet de modifier la taille de la zone mémoire(déjà réservé par malloc ou calloc) pointée par le pointeur p à N octets.
- Sa définition se trouve dans la bibliothèque standard `<stdlib.h>`
- Elle retourne un pointeur sur une zone mémoire de N octets.
- Elle retourne la valeur Null en cas d'echec.
- Elle préserve les premiers octets pointés par le pointeur p.



# Exercices

## Exercice1:

- Quelle est la valeur de la variable s après l'exécution de ce programme

```
#include<stdio.h>
main(){
int x[6]={1,2,3,4,5,6};
int i, s=0;
int*p=&x[0];
for(i=0;i<6;i++){
s+=*(p+i);}
}
```

## Exercices

### Solution :

- Les variables utilisées:  
x est un tableau de 6 entiers,  
k et s deux entiers et  
p est un pointeur vers un entier.  
La valeur initiale de s est 0.
- Le pointeur p va parcourir le tableau x en utilisant le pointeur p et calcule dans s la somme des éléments de x. On trouve  $s=21$

# Exercices

## Exercice 2:

- Écrire un programme qui réserve de la mémoire pour 10 int, 20 float et 5 double.
- Ecrire un programme qui réserve une zone mémoire pour un tableau de 10 nombres réels en simple précision initialisés à 0.

# Exercices

## Solution :

```
#include<stdio.h>
#include<stdlib.h>
main()
{int*p;
float*q;
Double*r;
p=(int*)malloc(10*sizeof(int));
q=(float*)malloc(10*sizeof(float));
r=(double*)malloc(10*sizeof(double));
}
```

# Exercices

## Solution :

```
#include<stdio.h>
#include<stdlib.h>
main()
{
float*p;
 p=(float*) calloc(10,sizeof(float))
}
```

# Exercices

## Exercice 3:

- Écrire un programme qui saisit un nombre de nombres entiers, les stocke dans un tableau, et ensuite, ajoute 3 autres nombres saisis et les stocke à la fin du premier tableau. Le programme doit en fin afficher tous éléments qui ont été saisi.



# Exercices

## Solution :

```
#include<stdio.h>
#include<stdlib.h>
main(){
int n;/* nombre d'entiers à saisir*/
int*tab;/*le tableau de stockage*/
int i;/*compteur*/
/*Demande du nombre d'entiers à saisir*/
printf("n=");
scanf("%d",&n);
/*on réserve l'espace pour le tableau*/
tab=(int*)malloc(n*sizeof(int));
/*on saisit les n entiers*/
```

## **Solution SUITE :**

```
for(i=0;i<n;i++)
```

```
{
```

```
printf("un entier:" ,i);
```

```
scanf("%d",&tab[i]);
```

```
}
```

/\*on réserve un espace de plus pour ajouter les 3 entiers qui seront saisis\*/

```
realloc(tab,(n+3)*sizeof(int));
```

```
for(i=n, i<n+3;i++)
```

```
{printf(" un entier:",i);
```

```
scanf("%d",&tab[i]);}
```

```
printf("on affiche toutes les valeurs de tab")
```

```
for(i=0;i<n+3;i++)
```

```
printf("%d\t,t",tab[i]);
```

```
}
```

## **Exercices**



# Les Fonctions

# La programmation modulaire

- Certains problèmes conduisent à des programmes longs, difficiles à écrire et à comprendre. On les découpe en des parties appelées **sous-programmes** ou **modules**
- Les modules sont des groupes d'instructions qui fournissent une solution à des parties bien définies d'un problème plus complexe. Ils ont plusieurs **intérêts** :
  - permettent de "**factoriser**" les **programmes**, càd de mettre en commun les parties qui se répètent
  - permettent une **structuration** et une **meilleure lisibilité** des programmes
  - **facilitent la maintenance** du code (il suffit de modifier une seule fois)
  - peuvent éventuellement être **réutilisées** dans d'autres programmes
- La structuration de programmes en sous-programmes se fait en C à l'aide des **fonctions**

# Fonctions

- On définit une fonction en dehors de la fonction principale main ( ) par:  
**type** nom\_fonction (**type1** arg1,..., **typeN** argN)  
  {  
    instructions constituant le corps de la fonction  
    return (expression)  
  }
- Dans la première ligne (appelée **en-tête de la fonction**):
  - **type** est le type du résultat retourné. Si la fonction n'a pas de résultat à retourner, elle est de type **void**.
  - le choix d'un nom de fonction doit respecter les mêmes règles que celles adoptées pour les noms de variables.
  - entre parenthèses, on spécifie les **arguments** de la fonction et leurs types. Si une fonction n'a pas de paramètres, on peut déclarer la liste des paramètres comme (**void**) ou simplement comme ()
- Pour fournir un résultat en quittant une fonction, on dispose de la commande **return**.

# Fonctions : exemples

- Une fonction qui calcule la somme de deux réels x et y :

```
double Som(double x, double y)
{
 return (x+y);
}
```

- Une fonction qui affiche la somme de deux réels x et y :

```
void AfficheSom(double x,
double y)
{
 printf (" %lf", x+y);
}
```

- Une fonction qui renvoie un entier saisi au clavier

```
int RenvoieEntier(void)
{
 int n;
 printf (" Entrez n \n");
 scanf (" %d ", &n);
 return n;
}
```

- Une fonction qui affiche les éléments d'un tableau d'entiers

```
void AfficheTab(int T[], int n)
{ int i;
 for(i=0;i<n;i++)
 printf (" %d \t", T[i]);
}
```



# Fonctions: Exercices

- Écrire :
- une fonction, nommée **affiche\_bonjour**, se contentant d'afficher « Bonjour tout le monde » (elle ne possède aucun argument ni valeur de retour).
- Une fonction, nommée **affiche\_somme**, qui affiche la somme de deux entiers (short) passés comme paramètres (elle ne possède aucune valeur de retour).
- Une fonction, nommée **produit**, qui reçoit en paramètre deux entiers (int) et retourne leur produit (int).
- Une fonction, nommée **imprime\_tab**, qui affiche les éléments d'un tableau de réels (float). Le tableau ainsi que le nombre d'éléments du tableau sont les paramètres de la fonction (elle ne possède aucune valeur de retour).

## Fonctions : Solutions

- `void affiche_bonjour(void) {  
 printf("Bonjour tout le monde \n"); }`
- `void affiche_somme(short a, short b) {  
 printf("%d",a+b); }`
- `int produit(int a, int b) {  
 return(a*b); }`
- `void imprime_tab( float tab[] , int nb_elements) {  
 int i;  
 for (i = 0; i < nb_elements; i++)  
 printf("%f \t",tab[i]); printf("\n"); }`

# Appel d'une fonction

- L'appel d'une fonction se fait par simple écriture de son nom avec la liste des paramètres : `nom_fonction (para1,..., paraN)`
- Lors de l'appel d'une fonction, les paramètres sont appelés **paramètres effectifs** : ils contiennent les valeurs pour effectuer le traitement. Lors de la définition, les paramètres sont appelés **paramètres formels**.
- L'ordre et les types des paramètres effectifs doivent correspondre à ceux des paramètres formels

- **Exemple d'appels:**

```
main()
```

```
{ double z;
 int A[5]= {1, 2, 3, 4, 5};
 z=Som(2.5, 7.3);
 AfficheTab(A,5);
}
```

## Appel d'une fonction

- Écrire une fonction, nommée puissance, qui calcule et retourne la valeur de  $x^n$ (double). Elle possède comme arguments x (double) et n (int).
- Écrire un programme C qui fait appel à la fonction puissance.

# Appel d'une fonction

```
#include<stdio.h>

double puissance(int n, double x) {
 double p = 1.0 ; // variable locale
 int i ; // variable locale
 for(i = 1 ; i <= n ; i++)
 p *= x ; //calcul de xn
 return p ; // valeur retournée }
```

```
Void main() {
 double y , z;
 scanf("%lf",&z); // appel de la fonction scanf définie dans <stdio.h>
 y = puissance(3, z) ; // appel de la fonction puissance
 printf("(%lf)^3 = %lf", z , y); // appel de la fonction printf<stdio.h> }
```



# Déclaration des fonctions

- Il est nécessaire pour le compilateur de connaître la définition d'une fonction au moment où elle est appelée. Si une fonction est définie après son premier appel (en particulier si elle est définie après main ), elle doit être **déclarée** auparavant.
- La déclaration d'une fonction se fait par son **prototype** qui indique les types de ses paramètres et celui de la fonction :  
**type nom\_fonction (type1,..., typeN)**
- Il est interdit en C de définir des fonctions à l'intérieur d'autres fonctions. En particulier, on doit définir les fonctions soit avant, soit après la fonction principale main.



## Déclaration des fonctions : exemple

```
#include<stdio.h>
```

```
float ValeurAbsolue(float); //prototype de la fonction ValeurAbsolue
```

```
main()
```

```
{ float x=-5.7,y;
```

```
 y= ValeurAbsolue(x);
```

```
 printf("La valeur absolue de %f est : %f \n " , x,y);
```

```
}
```

```
//Définition de la fonction ValeurAbsolue
```

```
float ValeurAbsolue(float a)
```

```
{
```

```
 if (a<0)
```

```
 a=-a;
```

```
 return a;
```

```
}
```

## Variables locales et globales

- Les variables manipulées dans un programme C ne sont pas toutes traitées de la même manière. En particulier, elles n'ont pas toutes la même durée de vie. On distingue deux catégories de variables : Variables globales et variables locales:
- 1. toute variable définie à l'extérieur des fonctions, d'un fichier source, est une variable globale: elle est connue et utilisable partout dans n'importe quelle fonction de ce fichier (sauf si elle est masquée par une variable locale). Elle est allouée dans la zone d'allocation statique(segment de données).

## Variables locales et globales

- 2. toute variable définie à l'intérieur d'une fonction est une variable locale. Elle n'est connue qu'à l'intérieur de la fonction dans laquelle elle est définie. Son contenu est perdu d'un appel à l'autre de la fonction. Elle est allouée dans le segment pile. Une variable locale cache la variable globale ayant même nom

# Variables locales et globales : remarques

- Les variables déclarées au début de la fonction principale main ne sont pas des variables globales, mais elles sont locales à main
- Une variable locale cache la variable globale qui a le même nom
- Il faut utiliser autant que possible des variables locales. Ceci permet d'économiser la mémoire et d'assurer l'indépendance de la fonction
- En C, une variable déclarée dans un bloc d'instructions est uniquement visible à l'intérieur de ce bloc. C'est une variable locale à ce bloc, elle cache toutes les variables du même nom des blocs qui l'entourent

# Variables locales et globales : exemple

```
#include<stdio.h>
```

```
int x = 7;
```

```
int f(int);
```

```
int g(int);
```

```
main()
```

```
{ printf("x = %d\t", x);
```

```
 { int x = 6; printf("x = %d\t", x); }
```

```
 printf("f(%d) = %d\t", x, f(x));
```

```
 printf("g(%d) = %d\t", x, g(x));
```

```
}
```

```
int f(int a) { int x = 9; return (a + x); }
```

```
int g(int a) { return (a * x) ; }
```

**Qu'affiche ce programme?**

# Variables locales et globales : exemple

```
#include<stdio.h>
```

```
void f(void);
```

```
int i;
```

```
main()
```

```
{ int k = 5;
```

```
 i=3;
```

```
 f();
```

```
 f();
```

```
 printf("i = %d et k=%d \n", i,k); }
```

```
void f(void) { int k = 1;
```

```
 printf("i = %d et k=%d \n", i,k);
```

```
 i++;
```

```
 k++;}
```

**Qu'affiche ce programme?**



## Variables locales et globales : exemple

- Il n'existe aucun lien entre la variable `k` de `main` et la variable `k` de `fct`.
- Toute modification de l'une n'a aucune influence sur l'autre.
- On peut avoir accès et modifier la variable `i`, qui est globale, à, partir de `main()` et `fct()`.
- La variable `k` de `fct()` est réinitialisée à 1 lors de chaque entrée dans `fct()`.

## Paramètres d'une fonction

- A l'appel d'une fonction avec paramètres, la valeur ou l'adresse du paramètre effectif est transmise au paramètre formel correspondant.
- **Passage par valeur:** Si le nom d'une variable (sauf le nom d'un tableau) apparaît dans l'appel d'une fonction, comme paramètre effectif, alors la fonction appelée reçoit la valeur de cette variable. Cette valeur sera recopiée dans le paramètre formel correspondant. Après l'appel de cette fonction, la valeur du paramètre effectif n'est pas modifiée.

## Paramètres d'une fonction

- **Passage par adresse:** Lorsqu'on veut qu'une fonction puisse modifier la valeur d'une variable passée comme paramètre effectif, il faut transmettre l'adresse de cette variable. La fonction appelée range l'adresse transmise dans une variable pointeur et la fonction travaille directement sur l'objet transmis. Un tableau est toujours passé par adresse puisque le nom d'un tableau est un pointeur constant (c'est-à-dire une adresse).

# Transmission des paramètres en C

- La transmission des paramètres en C se fait toujours par valeur
- Pour effectuer une transmission par adresse en C, on déclare le paramètre formel de type pointeur et lors d'un appel de la fonction, on envoie l'adresse et non la valeur du paramètre effectif
- Exemple : `void Increment (int x, int *y)`

```
{ x=x+1;
 *y =*y+1; }
main()
{ int n = 3, m=3;
 Increment (n, &m);
 printf("n = %d et m=%d \n", n,m); }
```

Résultat :

n=3 et m= 4

# Exercices

```
#include<stdio.h>
void fct_val(int) //passage d'argument par valeur
void fct_adr(int*) //passage d'argument par adresse
void main() {
 int i = 4 ;
 fct_val(i);
 printf("Après passage d'argument par valeur i : %d\n", i);
 fct_adr(&i);
 printf("Après passage d'argument par adresse i : %d\n", i);
}
```

```
void fct_val(int a) { a = a + 3; }
void fct_adr(int *a) { *a = *a + 3; }
```



# Exercices

Écrire un programme C qui saisie et affiche un tableau d'entiers en utilisant des fonctions

1. Permet de saisir la dimension du tableau: saisie\_N
2. Permet de saisir les éléments du tableau : saisie\_T
3. Permet d'afficher le tableau: affichage\_T



## Solution

```
#include<stdio.h>

void saisie_N(int *n) {
 printf("n (<=20) : ? ");
 scanf("%d", n); }

Void saisie_T(intT[] , int n) {
 int i ;
 for(i=0 ; i<n ; i++) {
 printf("%d\t",i);
 scanf("%d",&T[i]);}
}

Void affichage_T(intT[] , int n) {
 int i ;
 for(i=0 ; i<n ; i++)
 printf("%d\t",T[i]);}
```

```
int main() {
 int T[20] , N;
 saisie_N(&N);
 saisie_T(T , N);
 affichage_T(T , N);
 return 0;
}
```

## Exercices

Écrire un programme C qui saisie et affiche une matrice d'entiers courts de M ligne et N colonne en utilisant des fonctions

1. Permet de saisir la dimension du tableau: saisie\_M\_N
2. Permet de saisir les éléments du tableau : saisie\_A
3. Permet d'afficher le tableau: affichage\_A

# Solution

```
#include<stdio.h>

void saisie_M_N(short * m , short *n) {
printf("Entrer m (<=20) et n (<=30) : ? ") ;
scanf("%d", m , n); }

Void saisie_A(short B[][30] , int m , int n) {
 int i , j ;
 for(i = 0 ; i < m ; i++)
 for (j = 0 ; j < n ; j++)
 scanf("%d", &B[i][j]); }
```

# Solution

```
void affichage_A(short C[][30] , short m, short n) {
 int i , j ;
 for (i = 0 ; i < m ; i++) {
 for (j = 0 ; j < n ; j++)
 printf("%d\t", C[i][j]);
 printf("\n"); } }

int main() {
 short A[20][30] , M , N;
 saisie_M_N(&M , &N);
 saisie_A(A , M , N);
 affichage_A(A , M , N);
 return 0;
}
```

## Récurtivité

- Une fonction est réursive si elle contient dans sa définition un appel à elle même
- L'ordre de calcul est l'ordre inverse de l'appel de la fonction.
- Procédé pratique : Pour trouver une solution réursive d'un problème, on cherche à le décomposer en plusieurs sous-problèmes de même type, mais de taille inférieure. On procède de la manière suivante :
  - Rechercher un cas trivial et sa solution (évaluation sans récurtivité)
  - Décomposer le cas général en cas plus simples eux aussi décomposables pour aboutir au cas trivial.

# Récurtivité

- Exemple : Calcul du factorielle

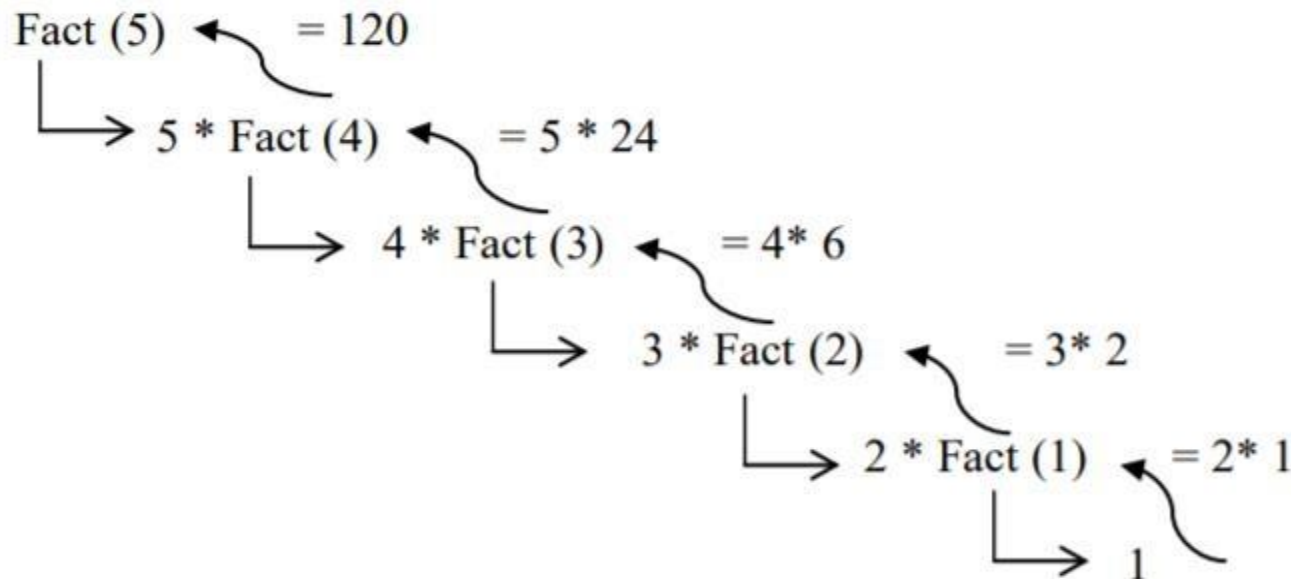
```
int fact (int n)
 { if (n==0) /*cas trivial*/
 return (1);
 else
 return (n* fact(n-1));
 }
```

Remarque : l'ordre de calcul est l'ordre inverse de l'appel de la fonction



# Récurtivité

- Prenons la fonction récursive de la factorielle et déroulons la pour  $N = 5$  : les flèches descendantes représentent les appels (de  $\text{Fact}(5)$  à  $\text{Fact}(1)$ ) on arrive alors à l'exécution de  $\text{fact}(1)$  pour laquelle la condition d'arrêt est valide alors le retour se fait à rebours (dans le sens contraire) et à chaque appel correspond alors la valeur associée.



## Fonctions récursives : exercice

1. Ecrire une fonction récursive qui calcule  $x^n$ ,  $x$  un réel et  $n$  un entier positif.
2. Ecrire une fonction récursive qui réalise le produit ( $x*y$ ) sachant que  $X$  et  $Y$  sont des entiers
3. Ecrire une fonction récursive qui calcule la valeur du  $n^{\text{ème}}$  terme de la suite de Fibonacci.

$$U(0)=U(1)=1$$

$$U(n)=U(n-1)+U(n-2)$$

## 1. Fonctions récursives : exercice

```
int puiss(float x, int n)
 { if (n==0) /*cas trivial*/
 return (1);
 else
 return (x* puiss(x, n-1));
 }
```

```
2.
int produit (int x, int y)
 { if (y==0) /*cas trivial*/
 return (0);
 else
 return (x+ produit(x, y-1));
 }
```

## Fonctions récursives : exercice

3.

```
int Fib (int n)
{ if (n==0 || n==1)
 return (1);
 else
 return (Fib(n-1)+Fib(n-2));

}
```

# Fonctions récursives

- Le processus récursif remplace en quelque sorte la boucle, c'est-à-dire un processus itératif.
- Il est à noter que l'on traite le problème à l'envers : on part du nombre, et on remonte à rebours jusqu'à 1, pour pouvoir calculer la factorielle par exemple.
- Cet effet de rebours est caractéristique de la programmation récursive.



**Types structures,  
unions et Types  
structures, unions et  
synonymes**



# Structures

- Une structure est un nouveau type de données constitué par un ensemble de variables (champs) qui peuvent être hétérogènes et de types différents
- La différence avec le type tableau est que les champs d'un tableau sont tous homogènes et du même type
- Les structures permettent de représenter des objets réels caractérisées par plusieurs informations, par exemple :
  - Une personne caractérisée par son nom (chaîne), son âge (entier), sa taille (réel), ...
  - Une voiture caractérisée par sa marque (chaîne), sa couleur (chaîne), son année modèle(entier), ...

## Type structure : Exemples

- Une date est un objet réel défini par : jour (entier ), mois (entier ou chaîne) et année(entier).
- Un nombre complexe est défini par sa parties réelle (réel) et sa partie imaginaire (réel).
- Un étudiant est défini par : nom (chaîne), prénom (chaîne), num\_CIN (chaîne), code (entier), num\_CNE (entier),....
- Un article est défini par : numéro (entier), libellé (chaîne), quantité en stock (entier), prix (réel).

## Déclaration d'une structure

- La déclaration d'une structure s'effectue en précisant le nom de la structure, ainsi que le nom et le type de ses champs :

- Syntaxe en C :

```
struct nom_structure {
 type 1 nom_champ1;
 type 2 nom_champ2;

 type N nom_champN ;
};
```

## Déclaration d'une structure

- Par l'intermédiaire du nom de la structure, on peut déclarer plusieurs variables de ce type de structure chaque fois que c'est nécessaire.
- Exemple :

```
struct Personne {
 char Nom[20];
 int Age;
 float taille;
};
```

Rq: Le nom d'une structure n'est pas un nom de variable, c'est le nom du type ou modèle de la structure

## Déclaration d'une variable structure

- La déclaration d'une structure ne réserve pas d'espace mémoire
- La réservation se fait quand on définit des variables correspondant à ce modèle de structure. Ceci peut se faire soit :

- après la déclaration de la structure, par exemple :

**struct Personne p1, \*p2, tab[10];**

//p2 est une variable de type pointeur sur une structure  
Personne

// tab est une variable de type tableau de 10 éléments  
(de type Personne)



# Déclaration d'une variable structure

- ou au moment de la déclaration de la structure

```
struct Personne {
 char Nom[20];
 int Age;
 float taille;
}; p1, *p2, tab[10];
```



# Exercices

- Déclarer Les structure suivantes:

**1.Un article** est défini par : numéro (entier : short), libellé(chaîne de 29 caractères), quantité en stock (entier : short), prix (réel : float).

**2.Un étudiant** est défini par : code (entier :short) ; nom (chaîne : 29) ; prénom (chaîne : 19) ; adresse (une adresse)

**Une adresse** est défini par : numéro de domicile (entier : short) ; nom de la rue ( chaîne : 29 ) code postale (entier : short) ; nom de la ville ( chaîne : 19) ; nom du pays (chaîne : 19).

# Exercices: solution 1

/\* Déclaration du type structure article\*/

```
struct article {
 short numero; // un numéro qui identifie l'article
 char libelle[30]; // le nom de l'article
 short qte_stock; //la quantité disponible en stock de l'article
 float prix; //le prix avec lequel est commercialisé l'article } ;
```

/\*Déclaration des variables du type structure article \*/

```
struct article art1 ; // art1 est une variable structure article
struct article art1 , art2; // art1 et art2 deux variables structure article
struct article *Pt_art; // Pt_art est une variable pointeur susceptible
de pointer une variable structure article
struct article tab_art[30] ; // tab_art est une variable tableau de 30
éléments de type structure article : tableau des articles
```

## Exercices: solution 2

```
 // type structure adresse
struct adresse {
 short n_domicile; /* numéro de la maison */
 char rue[30] ; /* nom de la rue */
 short code_postale; // le code postale
 char ville[20] ; /* nom de la ville */
 char pays[20] ; /* nom du pays */ } ;
```

## Exercices: solution 2

// type structure etudiant

```
struct etudiant{
 short code ; /* code de l'étudiant */
 char nom[30] ; /* nom de l'étudiant */
 char prenom[20] ; // prénom de l'étudiant
 struct adresse adr; // l'adresse de l'étudiant } ;
```

## Exercices: solution 2

/\*Déclaration des variables du type structure étudiant\*/

```
struct etudiant etd1 , edt2, // etd1 et edt2 deux variables
 *Pt_etd, // Pt_etd est une variable
pointeur susceptible de pointer une structure étudiant
 tab_etd[30] ; // tab_etd est tableau de 30
étudiants
```



## Initialisation d'une structure

- Lors de la déclaration d'une variable structure, on peut initialiser ses champs avec une notation semblable à celle utilisée pour les tableaux en indiquant la liste des valeurs respectives entre accolades.

- Exemple : `struct date`

```
{ unsigned short jour;
 char mois[10];
 unsigned short annee;
};
```

```
struct date d1= {15,"Novembre", 2013};
```



# Accès aux champs d'une structure

- L'accès à un champ d'une variable structure se fait par le nom de la variable suivi d'un point et du nom du champ

`nom_variable.nom_champ`

Exemple :

```
struct article art;
```

```
struct article
{
 short numero ;
 char libelle[30];
 short qte_stock;
 float prix;
};
```

```
/* Initialisation, depuis le clavier, des champs de la structure
art */
```

```
scanf("%d %d %f",& art.numero, & art.qte_stock,
&art.prix);
gets(art.libelle);
```

# Accès aux champs d'une structure

/\* Affichage du contenu de la structure art \*/

```
printf("Cette article a pour : \n");
printf("\t numéro: %d \n", art.numero);
printf("\t libellé: %s \n", art.libelle) ;
printf("\t quantité en stck: %d \n", art.qte_stock) ; printf("\t
prix: %f \n", art.prix) ;
```

## Accès à un champ via un pointeur de structure:

- Dans le cas d'une variable structure de type pointeur (ex : **struct Personne \*p2**), on utilise en général l'opérateur **->** pour accéder aux champs **nom\_variable -> nom\_champ** (ex: **p2->age** )

Exemple :

```
struct article *pt_art, art ;
pt_art= &art ;
```

/\* Initialisation, depuis le clavier, des champs de la structure art via la variable pointeur pt\_art\*/

```
scanf("%d %d %f",&(pt_art->numero) , &(pt_art->qte_stock),
&(pt_art->prix)) ;
gets(pt_art->libelle);
```

```
struct article
{
 short numero ;
 char libelle[30];
 short qte_stock;
 float prix;
};
```

## **Accès à un champ via un pointeur de structure:**

/\* Affichage du contenu de la structure art via la variable pointeur pt\_art\*/

```
printf("Cette article a pour : \n");
printf("\t numéro: %d \n", pt_art->numero);
printf("\t libellé: %s \n", pt_art->libelle) ;
printf("\t quantité en stck: %d \n", pt_art->qte_stock) ;
printf("\t prix: %f \n", pt_art->prix) ;
```

# Composition des structures

- Les structures peuvent être composées de champs de n'importe quel type connu: types de base, pointeur, tableau ou structure.
- Exemple de structure comportant un tableau et une structure :

```
struct Etudiant
```

```
{ int code;
```

```
 char Nom[20];
```

```
 struct date date_naissance;
```

```
 float notes[8]; // notes de l'étudiant dans 8 modules
```

```
} E1,E2;
```

- on peut écrire `E1.date_naissance.annee` pour accéder au champ `annee` de `E1.date_naissance` qui est de type `date`
- `E2.notes[3]` représente la note du module 4 de l'étudiant `E2`



# Opérations sur les variables structures

- Les structures sont manipulées en général champ par champ. Toutefois, la norme ANSI permet d'affecter une structure à une autre structure de même type, par exemple : `struct Etudiant E1,E2;`  
`E2=E1;` est une instruction valide qui recopie tous les champs de E1 dans les champs de E2
- Il n'est pas possible de comparer deux structures. Les instructions `if(E1==E2)` ou `if(E1!=E2)` ne sont pas permises
- L'opérateur `&` permet de récupérer l'adresse d'une variable structure (ex : `struct Personne p1, *p2;` `p2=&p1`)

Exemple:

```
struct article art= {1, "Ecran TFT 19" ,12 , 2500.0} , *pt_art ; pt_art =
&art ;
```

```
pt_art->qte_stck = 10; // On modifie la quantité en stock
```

```
pt_art->prix = 2000.0; //On modifie le prix : l'article est en promotion
```



# Opérations sur les variables structures

- L'opérateur sizeof permet de récupérer la taille d'un type structure ou d'une variable structure (ex : sizeof (struct Personne) ou sizeof (p1) )

Exemple:

```
struct article art;
printf("taille en octets de la structure article : %d\n", sizeof(article)) ;
/* ou */
printf("taille en octets de la structure article : %d\n", sizeof(struct
article)) ;
```

# Structures et fonctions

Une structure peut être utilisée comme argument d'une fonction et transmise par valeur (ou par adresse via un pointeur)

## Exemple de transmission par valeur

```
struct couple { int a;
 float b;};
void zero (struct couple s)
{ s.a=0; s.b=0;
 printf(" %d %f \n ", s.a, s.b);
}
main()
{ struct couple x;
 x.a=1;x.b=2.3;
 printf("avant: %d %f \n ", x.a, x.b);
 zero(x);
 printf("après: %d %f \n ", x.a, x.b);
}
```

## Exemple de transmission par adresse

```
struct couple { int a;
 float b;};
void zero (struct couple *s)
{ s->a=0; s->b=0;
 printf(" %d %f \n ", s->a, s->b);
}
main()
{ struct couple x;
 x.a=1;x.b=2.3;
 printf("avant: %d %f \n ", x.a, x.b);
 zero(&x);
 printf("après: %d %f \n ", x.a, x.b);
}
```

## Définition de types synonymes: typedef

- En C, on peut définir des types nouveaux synonymes de types existants (simples, pointeur, tableau, structure,...) en utilisant le mot clé **typedef**. Ces nouveaux types peuvent être utilisées ensuite comme les types prédéfinis
- **Exemple d'un type synonyme d'un type simple :**
- **typedef int entier;** définit un nouveau type appelé **entier** synonyme du type prédéfini **int**
- **entier i=4,T[10]** ; le type entier est utilisé ici pour déclarer des variables
- Remarque : l'intérêt de typedef pour les types de base est limité puisqu'il remplace simplement un nom par un autre

# Définition de types synonymes: typedef

**Exemple d'un type synonyme d'un type pointeur :**

```
typedef int *ptr_entier;
```

```
ptr_entier p1,p2; p1 et p2 sont des pointeurs sur des int
```

```
typedef char *chaine;
chaine ch;
```

- **Type synonyme d'un type tableau :**

```
typedef float tableau[10];
```

```
tableau T; T est un tableau de 10 réels
```

```
typedef int matrice[4][5];
```

```
matrice A; A est une matrice d'entiers de 4 lignes et 5
colonnes
```

## Exemples de typedef

- **Type synonyme d'un type structure :**

```
typedef struct
{ int jour;
 int mois;
 int annee;
} date;
```

- date est le nom du nouveau type et non d'une variable
- On peut utiliser directement ce type, par exemple: **date d, \*p;**
- Ceci permet simplement d'éviter l'emploi du mot clé struct dans les déclarations de variables



# Structures récursives

- Une structure est dite récursive si elle contient des pointeurs vers elle même, par exemple :

```
struct Individu
{ char*Nom;
 int Age;
 struct Individu *Pere, *Mere;
};
```

```
struct noeud
{ int val;
 struct noeud *suivant;
};
```

- Ce genre de structures est fondamental en programmation car il permet implémenter la plupart des structures de données employées en informatique